

# Fault-tolerant Quantum Computation based on Surface Code

---

Yiming Zhang

2025-06-13

The University of Science and Technology of China

|                                   |    |
|-----------------------------------|----|
| 1. Quantum Error Correction ..... | 2  |
| 2. Surface Code Computation ..... | 16 |
| 3. Surface Code Compilation ..... | 36 |

# 1. Quantum Error Correction

---

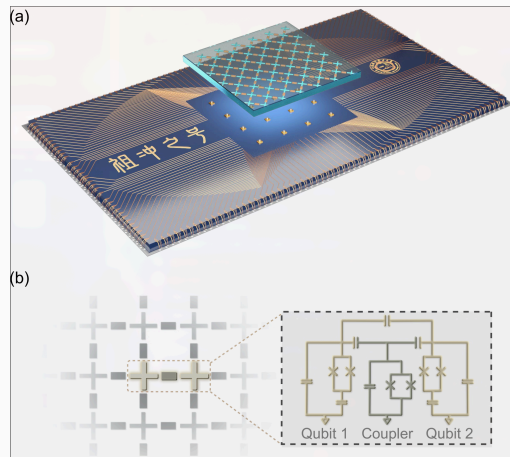
# The Promise of Quantum Computer

*Quantum Computers* are expected to solve certain exponentially hard classical problems in polynomial time or provide superior speedup for some applications[1]:

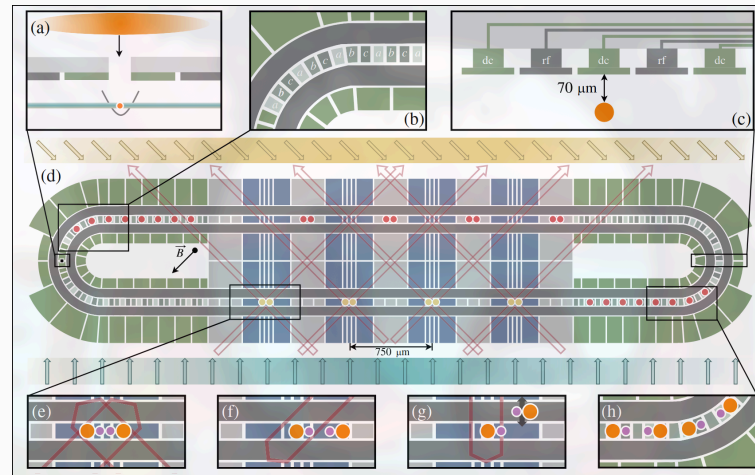
- *Condensed Matter Physics*: simulation of Fermi-Hubbard model, SYK model, etc.
- *Quantum Chemistry*: computing electronic structure and vibrational structures.
- *Optimization*: combinatorial optimization and continuous optimization problems.
- *Cryptography*: breaking of RSA, ECC, and symmetric key cryptography.



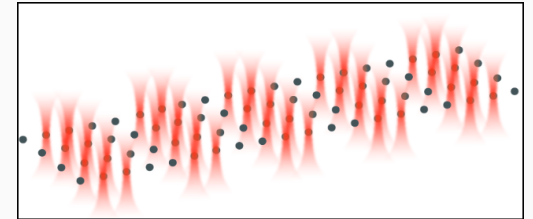
# Build a Quantum Computer



Superconducting Qubits



Trapped Ions

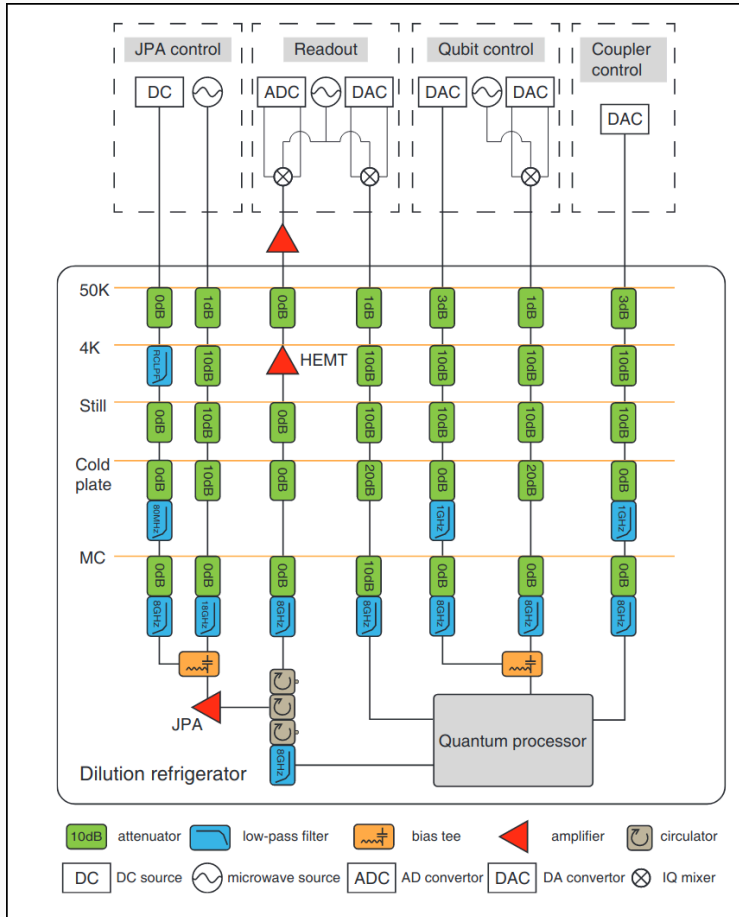


Neutral Atoms



Topological Qubits

# Superconducting Quantum Computer



- *Bottom: Quantum Processor(QPU)*
  - aluminum film on sapphire substrate
  - MBE, optical lithography, etc.
  - flip-chip bonding, TSV, etc.
  - dilution refrigerator
- *Middle: Control Electronics*
  - DAC, ADC, AWG, attenuator, filter, amplifier, etc.
- *Top: Control Software System*
  - abstraction of hardware details
  - quantum ISA (QCIS)
  - calibration and experiments

Quantum systems are extremely sensitive to their environment, the interaction with the environment, unexpected coupling between qubits, and nonideal control can lead to unwanted evolution of quantum states, which can be characterized by a *quantum channel*:

$$\mathcal{E}(\rho) = \sum_k A_k \rho A_k^\dagger$$

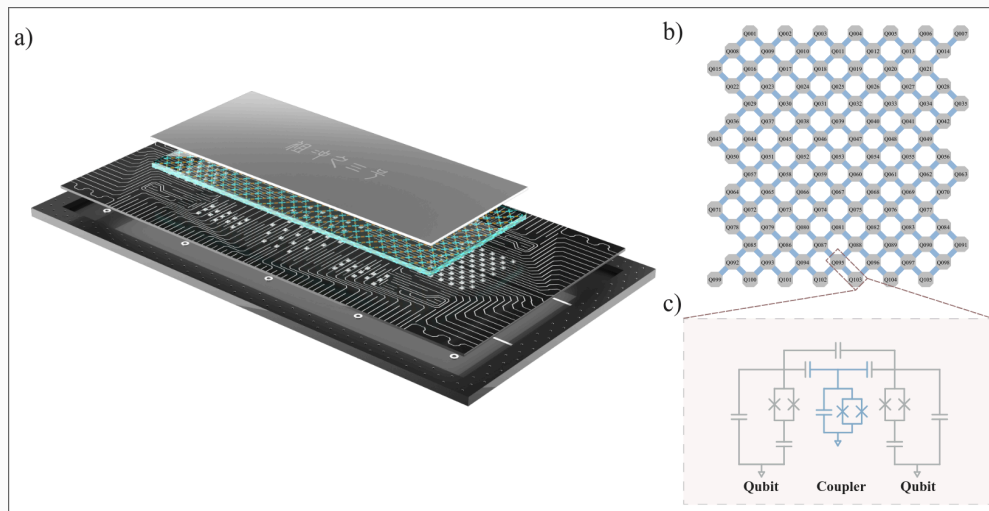
*Dephasing Channel:*  $\mathcal{E}(\rho) = (1 - p)\rho + pZ\rho Z^\dagger$

*Amplitude Damping:*  $\mathcal{E}(\rho) = A_0 \rho A_0^\dagger + A_1 \rho A_1^\dagger$ ,  $A_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-p} \end{pmatrix}$ ,  $A_1 = \begin{pmatrix} 0 & \sqrt{p} \\ 0 & 0 \end{pmatrix}$

*Depolarizing Channel:*  $\mathcal{E}(\rho) = (1 - p)\rho + \frac{p}{3}(X\rho X^\dagger + Y\rho Y^\dagger + Z\rho Z^\dagger)$

And more: Unitary Channel, Leakage Channel, Erasure Channel, etc.

# The limitations of NISQ

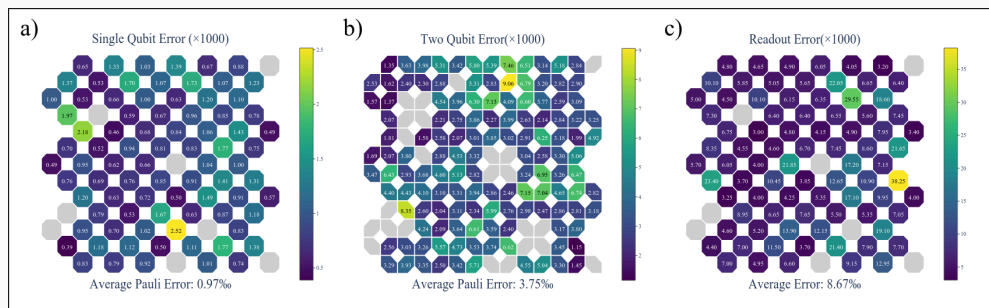


- *Quantity*

- ▶ 105 transmon qubits
- ▶ 182 tunable couplers

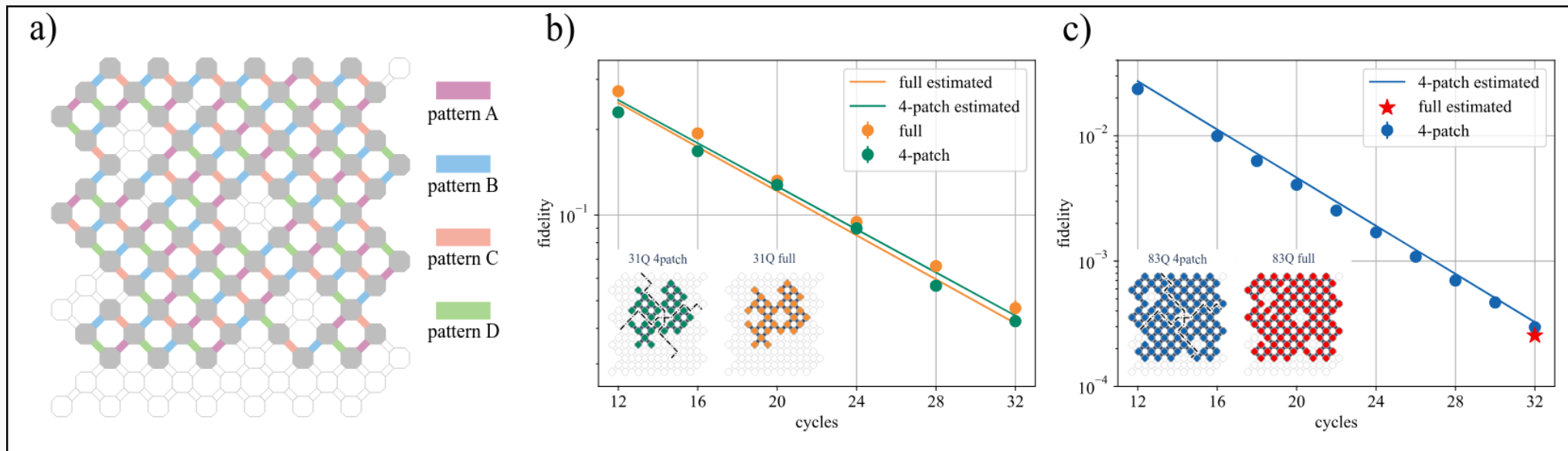
- *Quality (benchmarked with parallel XEB)*

- ▶ 99.90% single-qubit gate
- ▶ 99.62% two-qubit gate(FSIM)
- ▶ 99.18% readout
- ▶  $72\mu s$  relaxation time( $T_1$ )
- ▶  $58\mu s$  dephasing time( $T_2^{\text{CPMG}}$ )



# The limitations of NISQ

*Random Circuit Sampling*: demonstration of quantum advantage in NISQ on a carefully designed task that is of no practical use[2], [3].

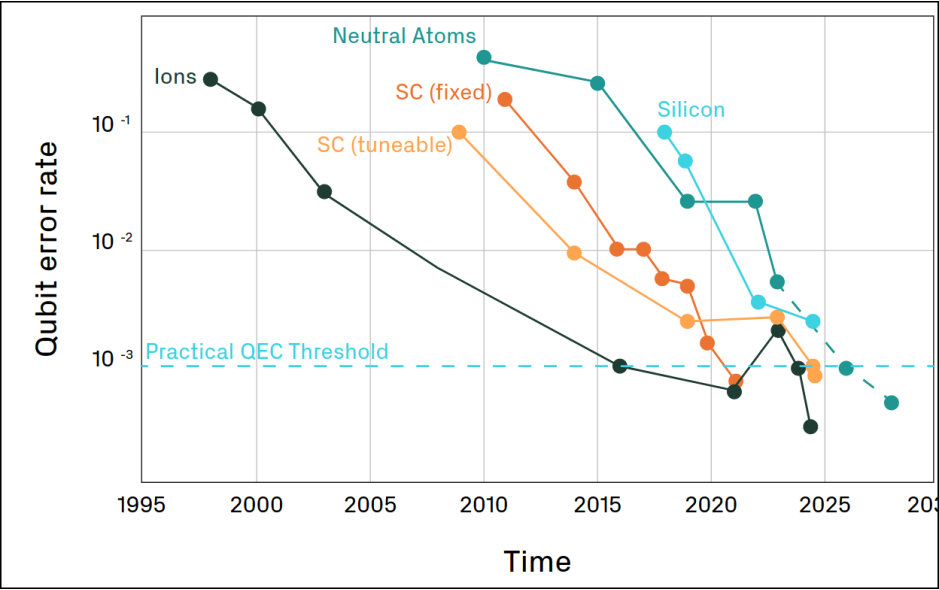


It is a benchmark for the NISQ hardware rather than a quantum application.

# Practical Quantum Advantage

*Practical quantum advantage* requires qubits with ultra-high fidelity ( $10^{-9} \sim 10^{-15}$ ) and might run for weeks or more on quantum hardware[4], [5].

| Application<br>(One shot of QPE)                                                    | Logical qubits<br>N | Depth<br>D  | QuOps<br>(lower bound)<br>N.D |
|-------------------------------------------------------------------------------------|---------------------|-------------|-------------------------------|
| Hubbard model (8x8)<br>Campbell<br>Quantum Sci. Technol., 7 015007<br>(2021)        | 162                 | 5.8 million | 0.9 GigaQuOp                  |
| Jellium / Free electron gas (5x5x5)<br>Kivlichan et. al.<br>Quantum, 4, 296 (2020). | 378                 | 89 million  | 34 GigaQuOp                   |
| RSA with 2024 bits<br>Ekerä and Gidney<br>Quantum, 5, 433 (2021).                   | 6,115               | 2.1 billion | 13 TeraQuOp                   |
| FeMoCo (Li orbitals / THC method)<br>Lee et. al.<br>PRX Quantum, 2, 030305 (2021).  | 2,196               | 130 billion | 285 TeraQuOp                  |



It’s not expected to achieve this level of fidelity by improving the hardware.

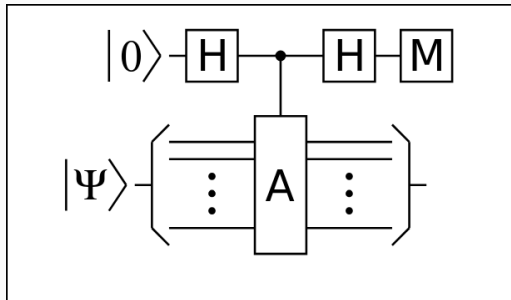
# Quantum Error Correction(QEC)

1. **Encoding** information with redundancy:

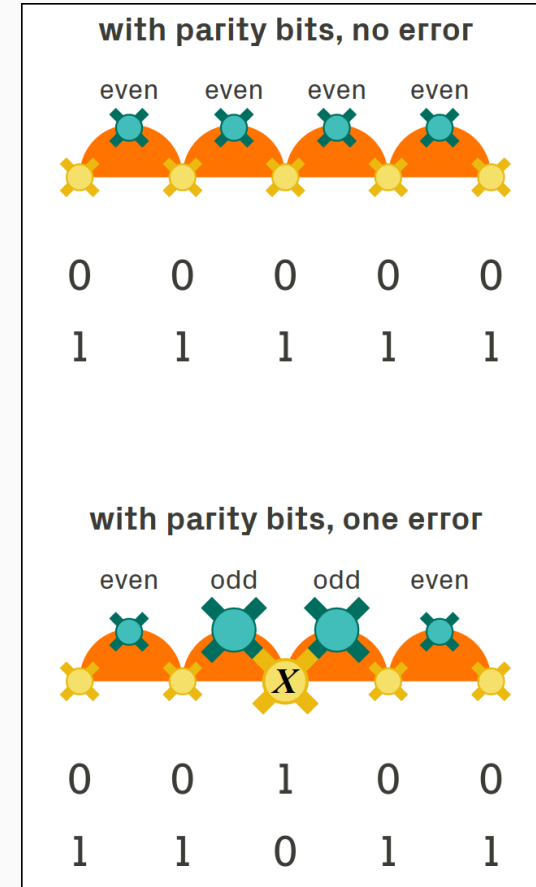
$$|0\rangle_L = \frac{1}{\sqrt{2}} |00000\rangle + |11111\rangle,$$

$$|1\rangle_L = \frac{1}{\sqrt{2}} |00000\rangle + |11111\rangle,$$

2. **Detecting** errors by projection measurements:



3. **Decoding** and correcting errors.



A compact representation of quantum states with (signed) Pauli operators:

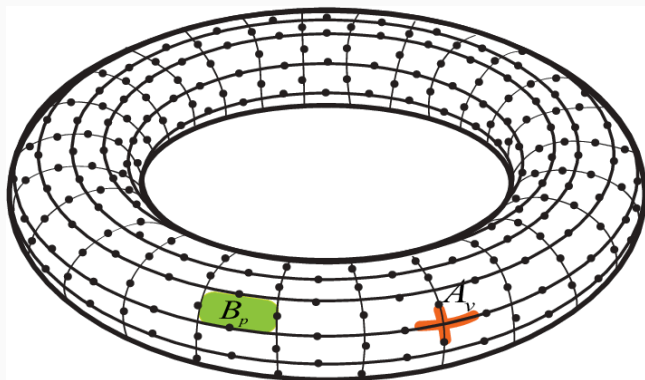
$$|\Psi\rangle = \frac{1}{\sqrt{2}} |000\rangle + |111\rangle \Rightarrow \langle +XXX, +ZZI, +IZZ \rangle$$

The state  $|\Psi\rangle$  is *stabilized* by the 3 independent stabilizers.

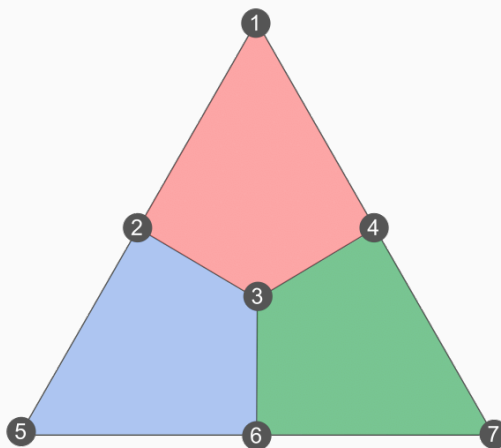
For complex states, it's tedious or even impossible to write down the full state vector. The stabilizer representation comes to rescue.



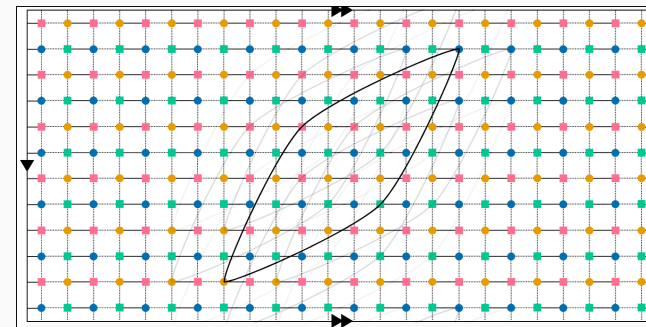
# Quantum Error Correction Codes(QECC)



Toric Code



Color Code



BB Code(QLDPC)

And more: Floquet codes, Subsystem codes, Bosonic codes, ...

# Topological Quantum Error Correcting Codes

- *Stabilizer Formalism* of the code:

$$S_p^X = \prod_{j \in \partial p} X_j, S_p^Z = \prod_{j \in \partial p} Z_j$$

- *Hamiltonian system* whose low-energy excitations are anyons:

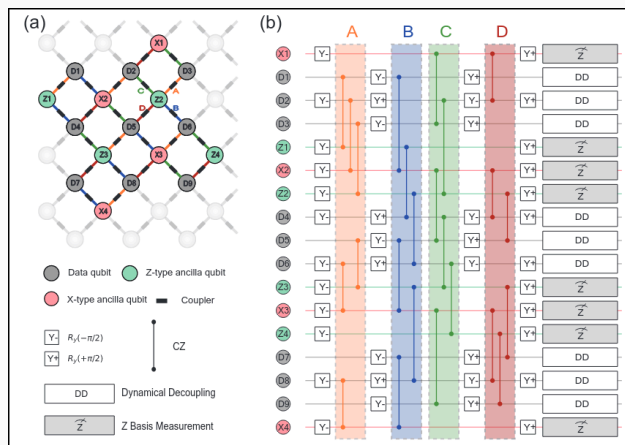
$$H = - \sum_{p \in P} S_p^X - \sum_{p \in P} S_p^Z$$

- *Anyon Model* describing the anyon excitations of the system:

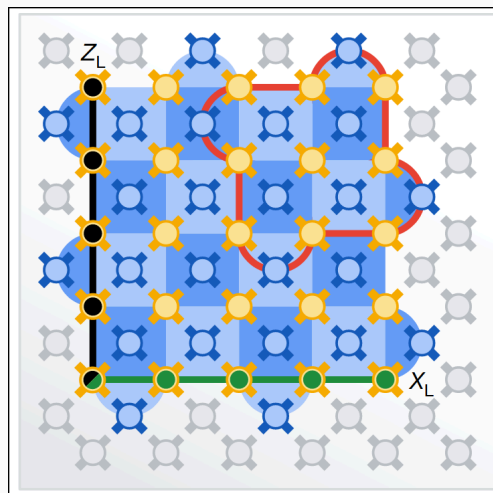
|      |      |      |
|------|------|------|
| $rx$ | $gx$ | $bx$ |
| $ry$ | $gy$ | $by$ |
| $rz$ | $gz$ | $bz$ |

- *Symmetries* between the anyonic excitations give rise to non-trivial domain walls and pointlike topological defects that can be utilized for fault-tolerant quantum protocols.

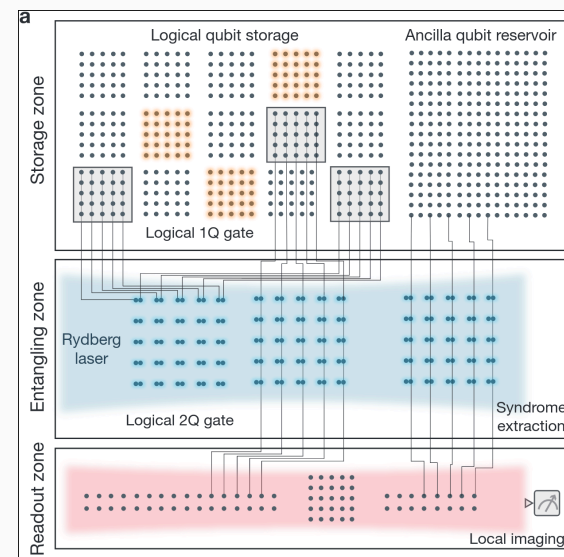
# Experimental Progress on QEC



$d = 3$  Surface Code  
USTC 2022[6]

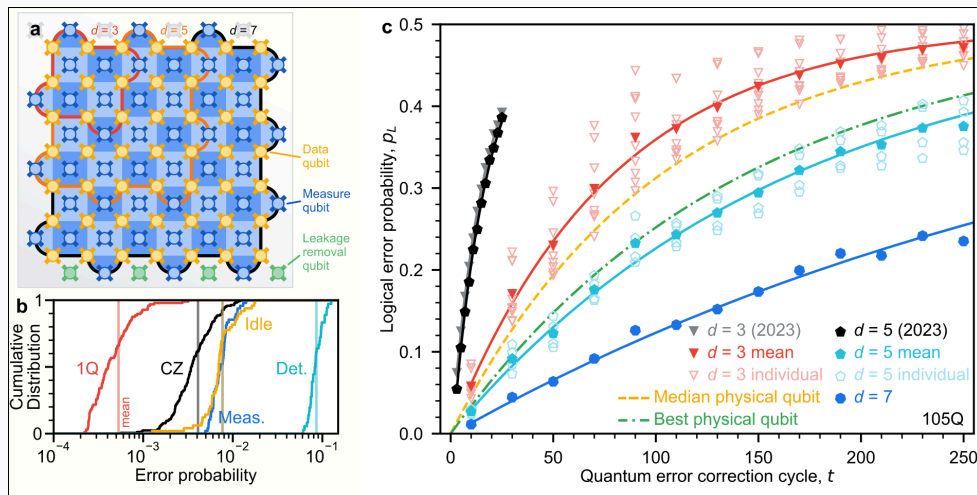


$d = 3, 5$  Surface Code  
Google 2023[7]



280 Physical Qubits  
40 Logical Qubits  
QuEra 2023[8]

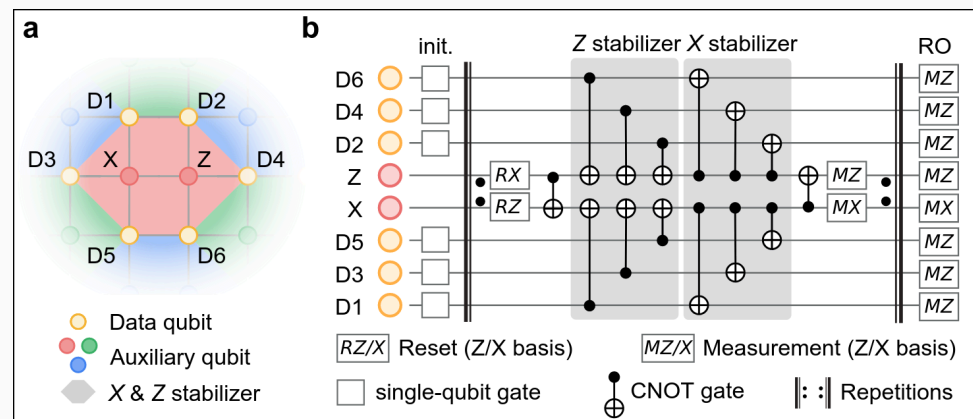
# Experimental Progress on QEC



$d = 3, 5, 7$  Surface Code

**Below Threshold**

Google 2024[9]



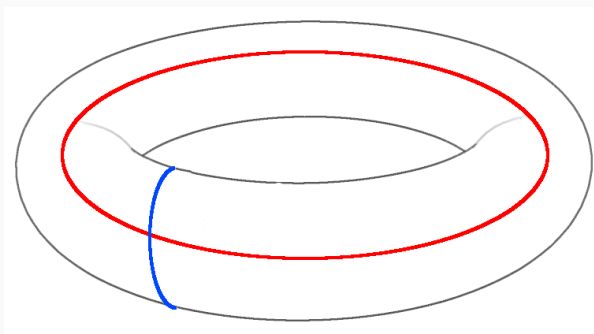
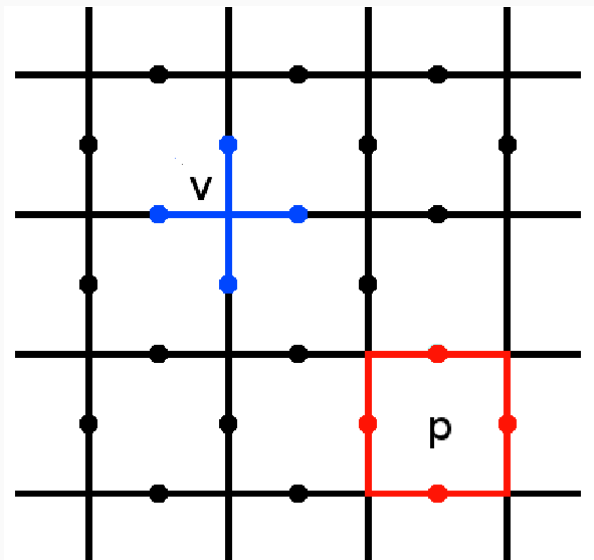
$d = 3, 5$  Color Code

Google 2024[10]

## 2. Surface Code Computation

---

# Toric Code



Define stabilizers on a 2D square lattice:

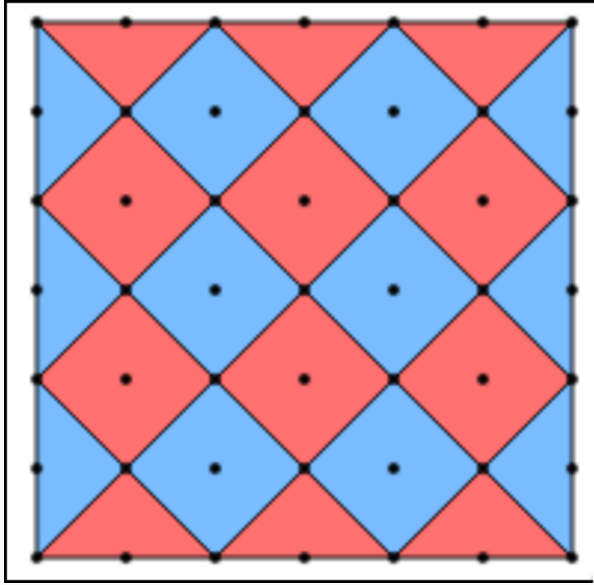
$$A_v = \prod_{i \in v} X_i, B_f = \prod_{i \in f} Z_i$$

Hamiltonian:

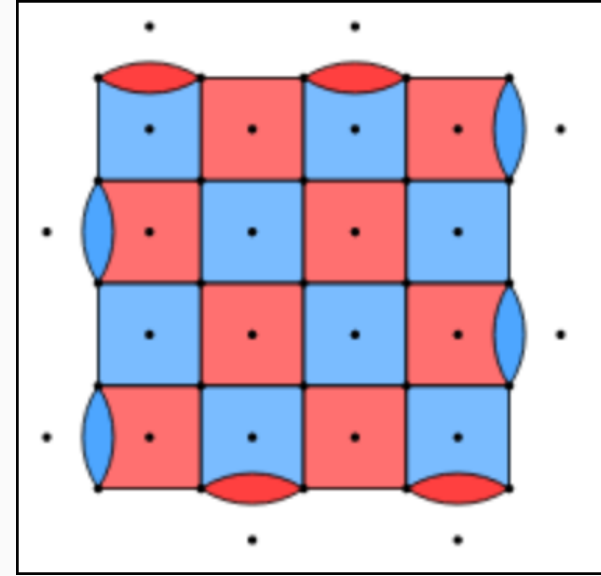
$$H = - \sum_{v \in V} A_v - \sum_{f \in F} B_f$$

Nontrivial loops define logical operators.  
Encodes two logical qubits in the Hilbert space:  $[[2L^2, L, 2]]$ .

# Surface Code

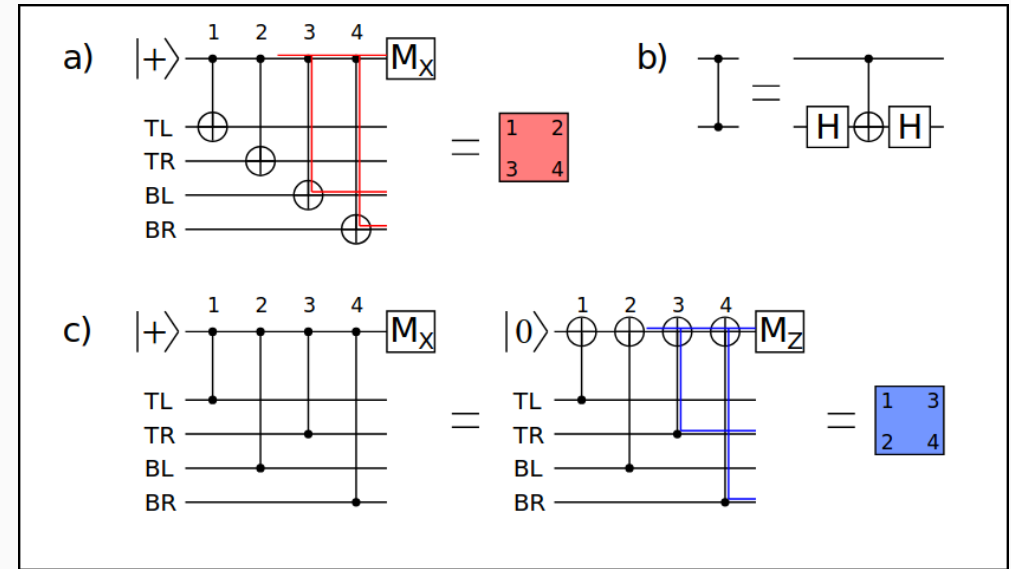
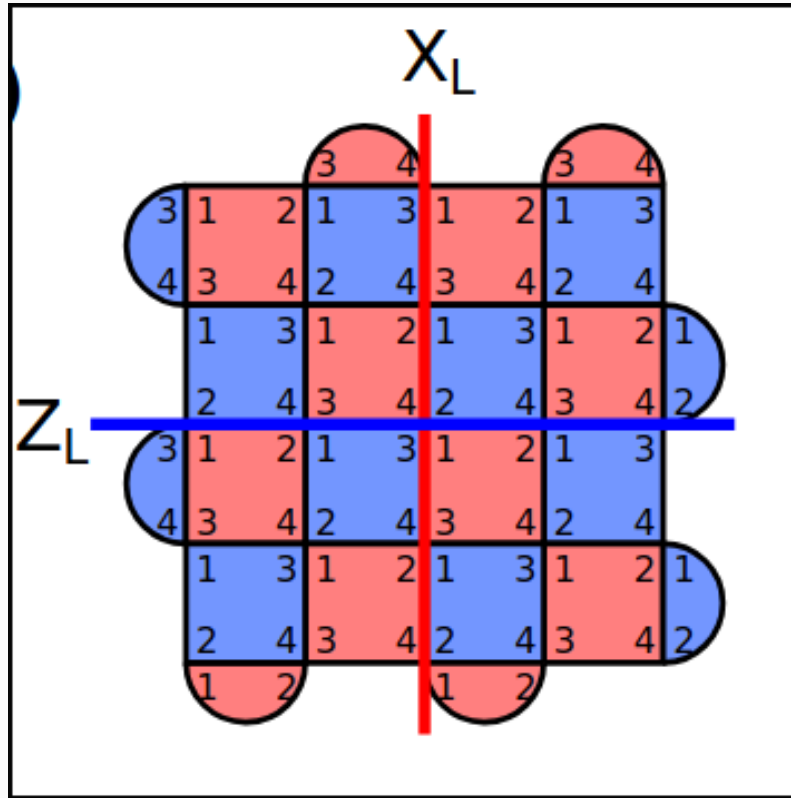


Unrotated Surface Code  
 $[[2d^2 - 2d + 1, 1, d]]$



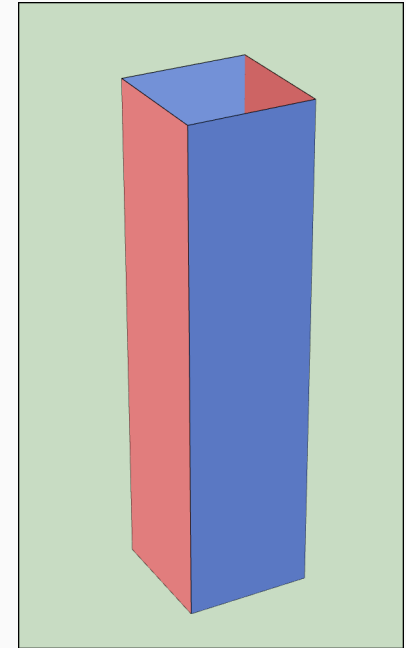
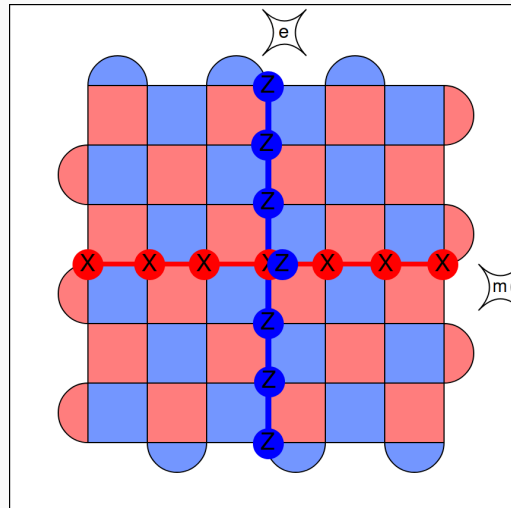
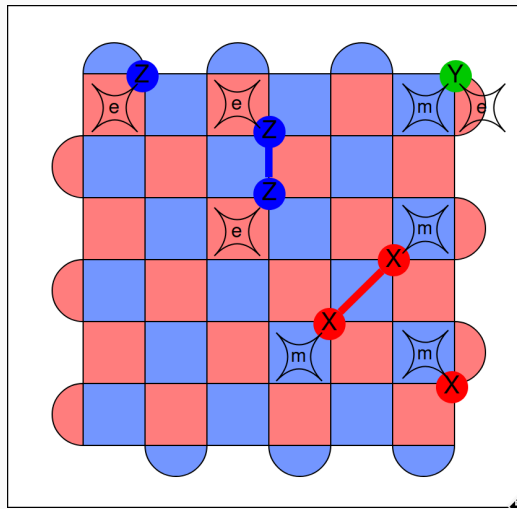
Rotated Surface Code  
 $[[2d^2 - 1, 1, d]]$

# Stabilizer Measurements

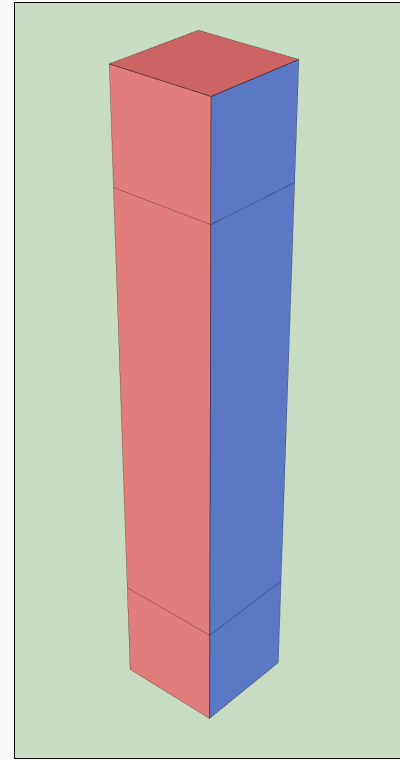
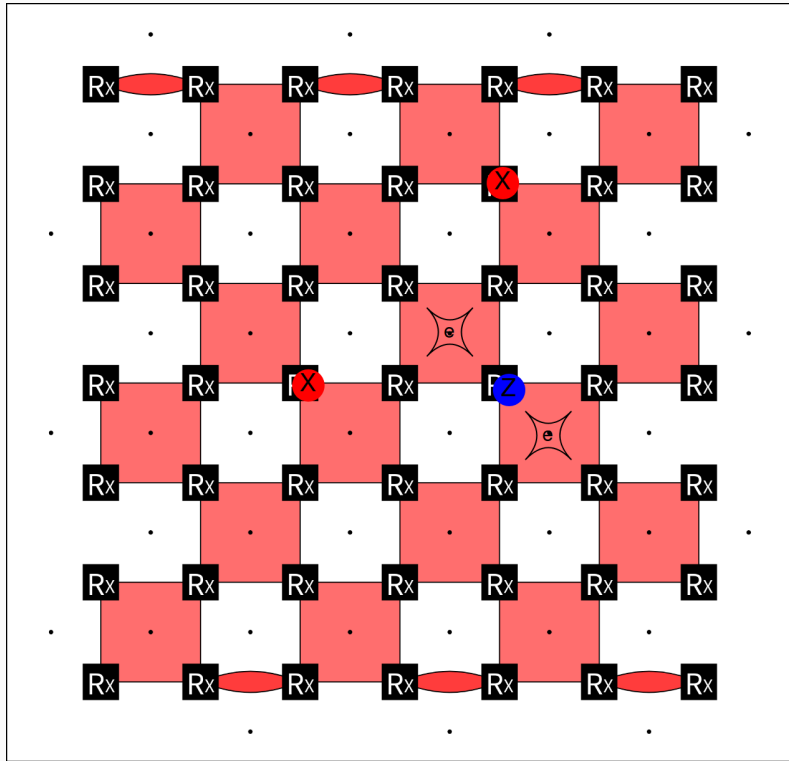




# Anyonic Excitations in Surface Code



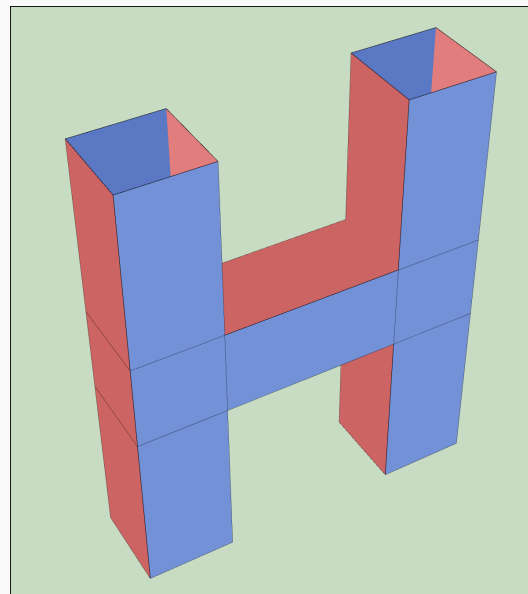
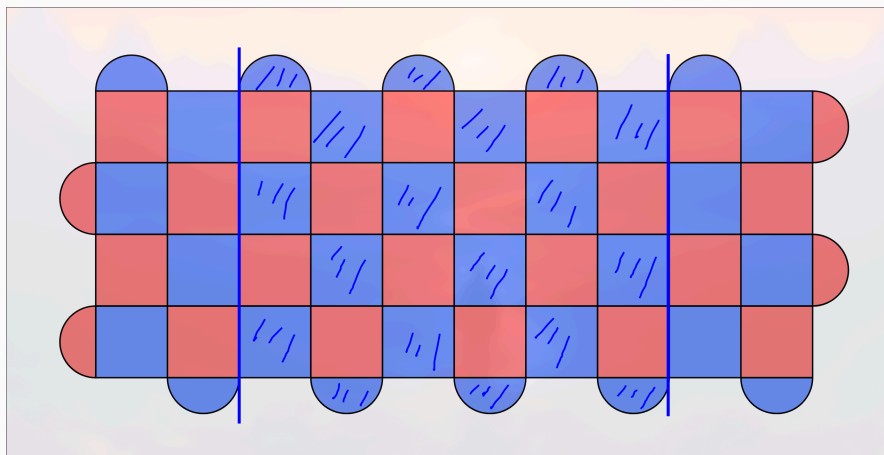
# Time Boundaries



# Lattice Surgery

*Lattice Surgery*[11] connects the boundaries of surface codes and **correlates the logical operators** in spacetime.

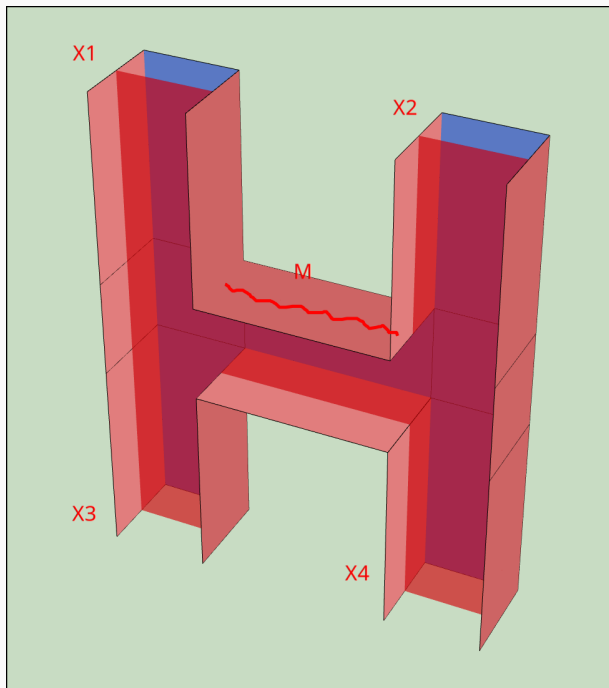
$O(2d \times d \times d)$  spacetime volume



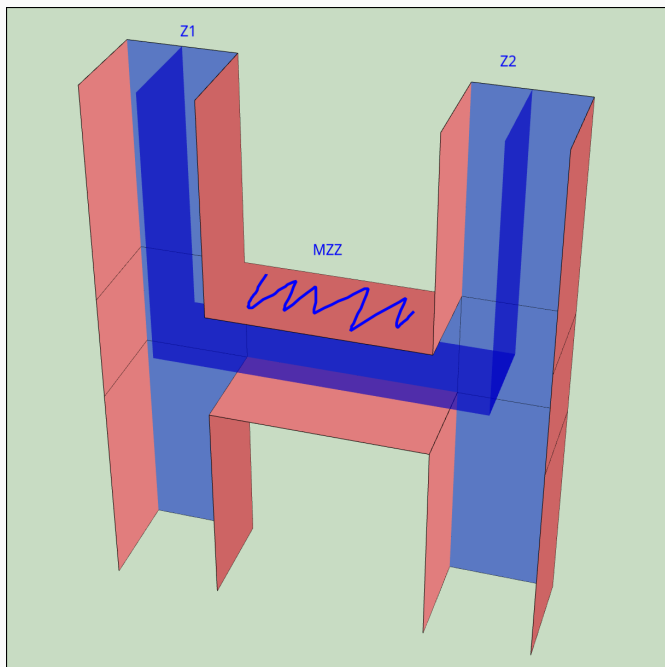
We can fault-tolerantly implement  $M_{ZZ}$  and  $M_{XX}$  parity measurements for surface codes with lattice surgery. They are the true primitives for fault-tolerant surface code computation instead of CNOT/CZ gates. It's kind of MBQC-style computation.

# Correlation Surface

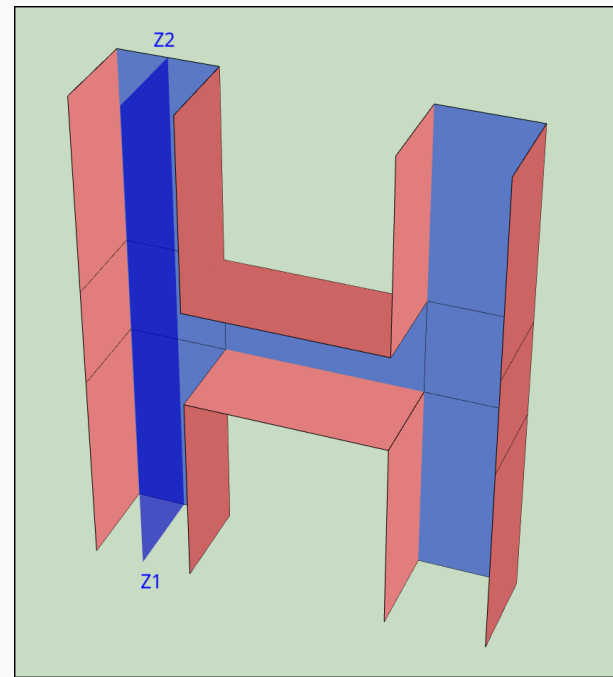
*Correlation Surface* is a set of 2D surfaces in spacetime that represents the correlation between logical operators.



$$X_1 X_2 X_3 X_4 = (-1)^M$$



$$Z_1 Z_2 = (-1)^{M_{ZZ}}$$



$$Z_1 Z_2 = +1$$

zero spacetime volume

Physically we can apply the logical operators  $X_L/Z_L$  to implement the logical Pauli gates:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

In practice, we track a *Pauli Frame* in software to keep track of the logical Pauli gates applied. We reinterpret the subsequent logical measurement outcomes based on the Pauli Frame at that time. Therefore, Pauli gates are zero-cost operations.

# Universal Gateset: $H$

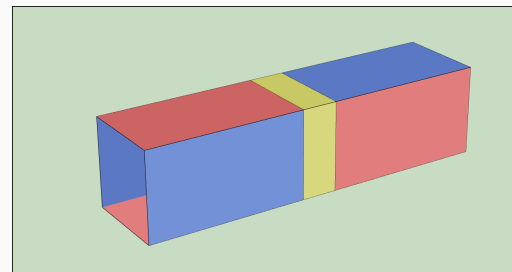
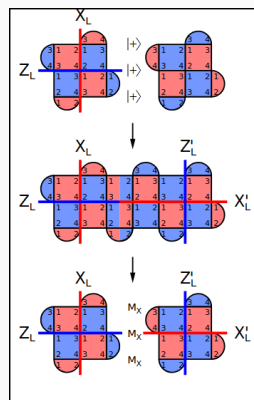
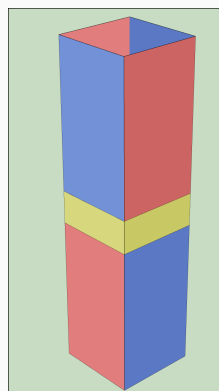
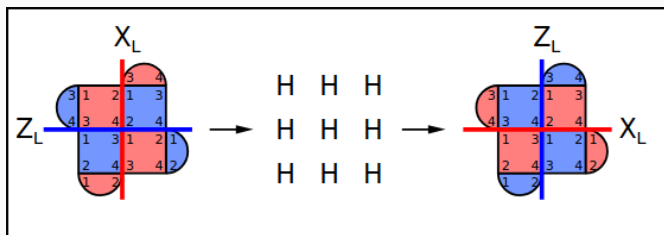
Hadamard Gate:

$O(d^2 \times 1)$  spacetime volume

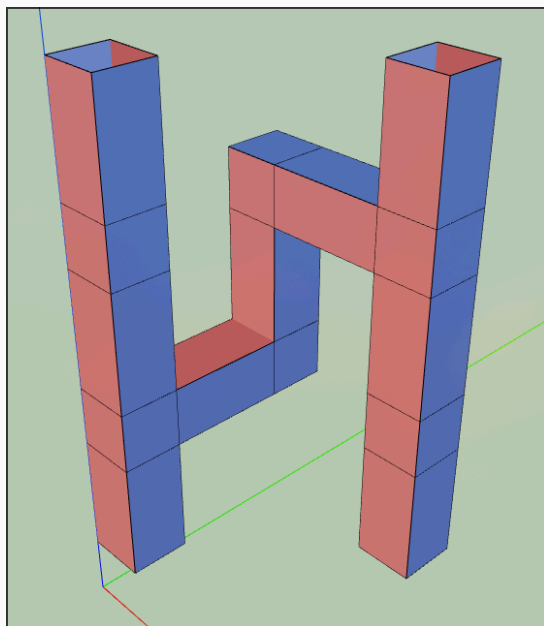
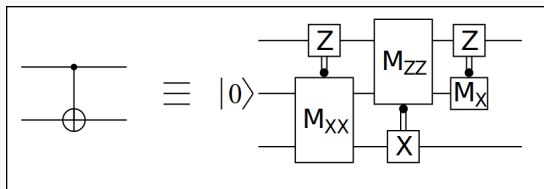
$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad X \xleftrightarrow{H} Z$$

We can construct a *transparent domain wall* in the bulk of the surface code to realize the anyon mapping:

$$e \xleftrightarrow{\varphi} m$$



# Universal Gateset: CNOT

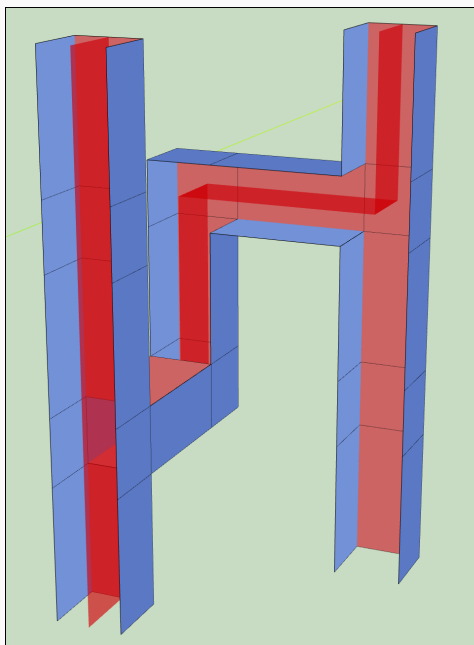


$$XI \xrightarrow{\text{CNOT}} XX,$$

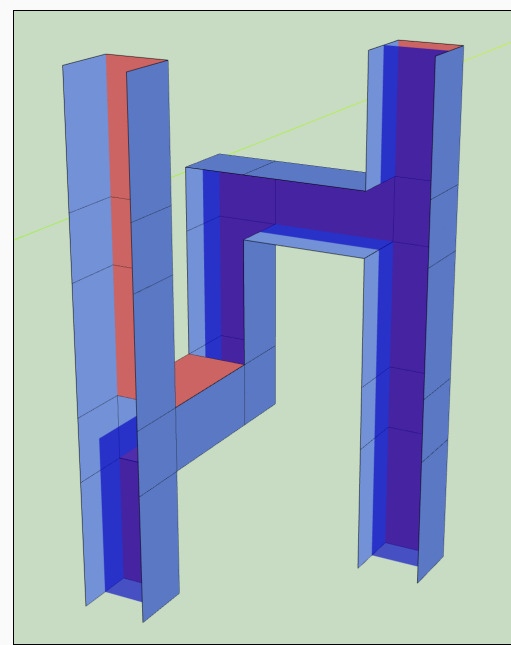
$$IX \xrightarrow{\text{CNOT}} IX$$

$$ZI \xrightarrow{\text{CNOT}} ZI,$$

$$ZZ \xrightarrow{\text{CNOT}} IZ$$



$$XI \longrightarrow XX$$

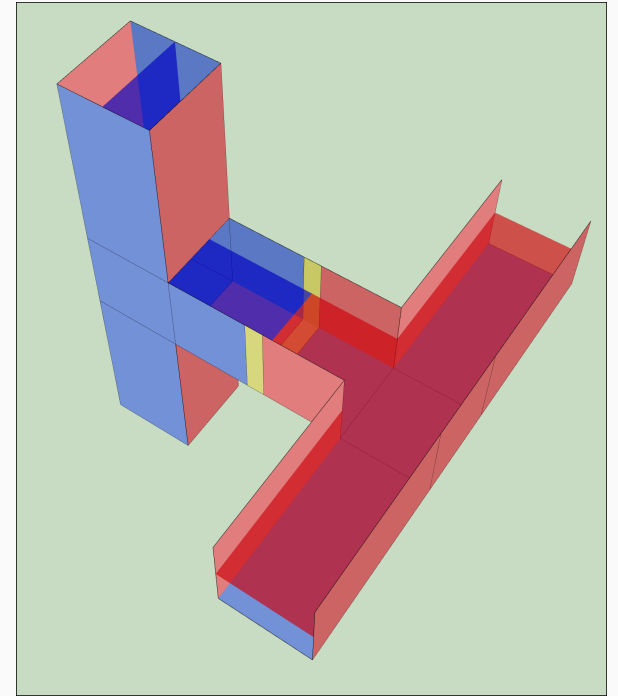
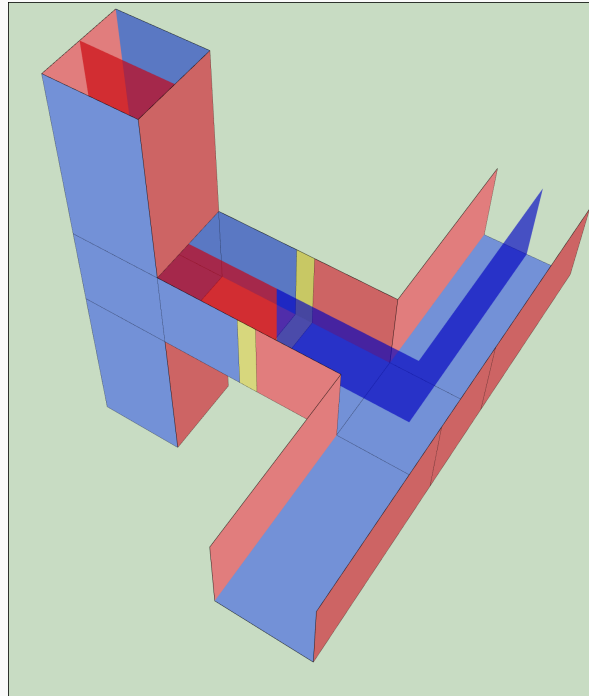
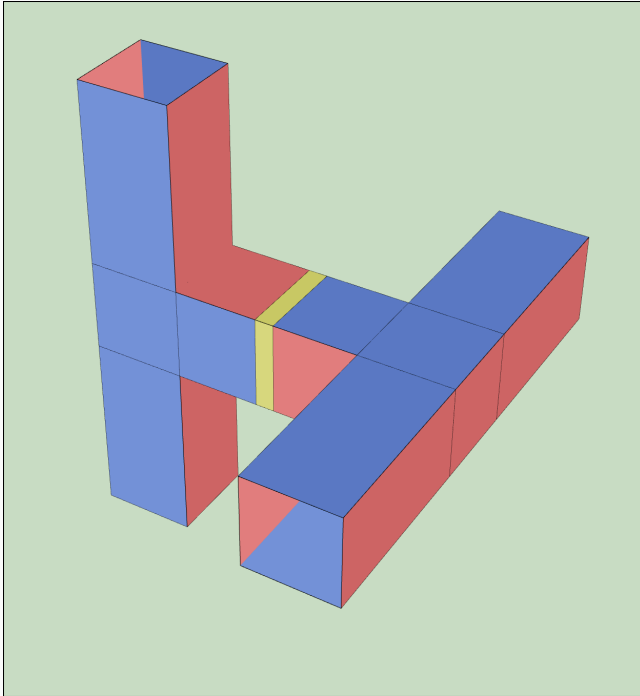


$$ZZ \longrightarrow IZ$$

$$O(3d^2 \times 2d) \text{ spacetime volume}$$

# Universal Gateset: CZ

$$\begin{aligned} XI &\xrightarrow{\text{CZ}} XZ, & IX &\xrightarrow{\text{CZ}} ZX \\ ZI &\xrightarrow{\text{CZ}} ZI, & IZ &\xrightarrow{\text{CZ}} IZ \end{aligned}$$



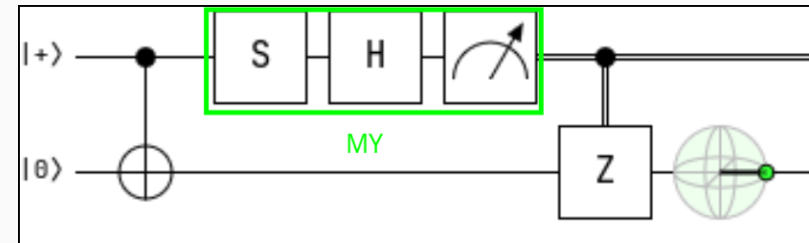
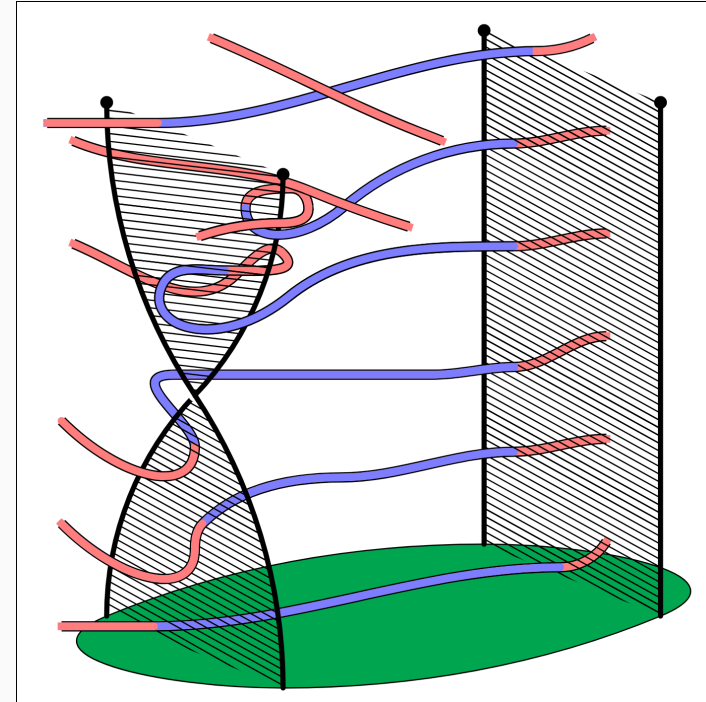


# Universal Gateset: $S$

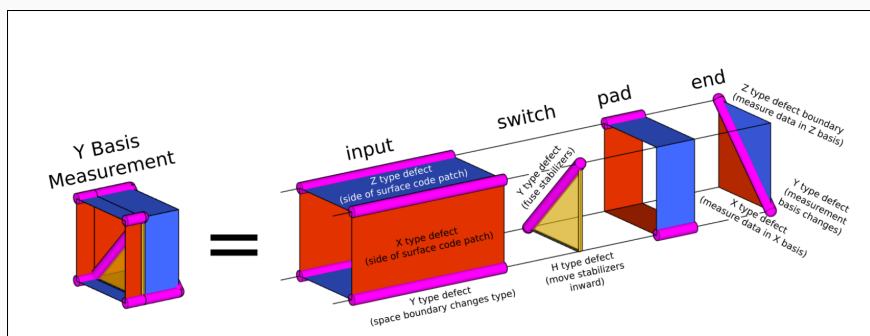
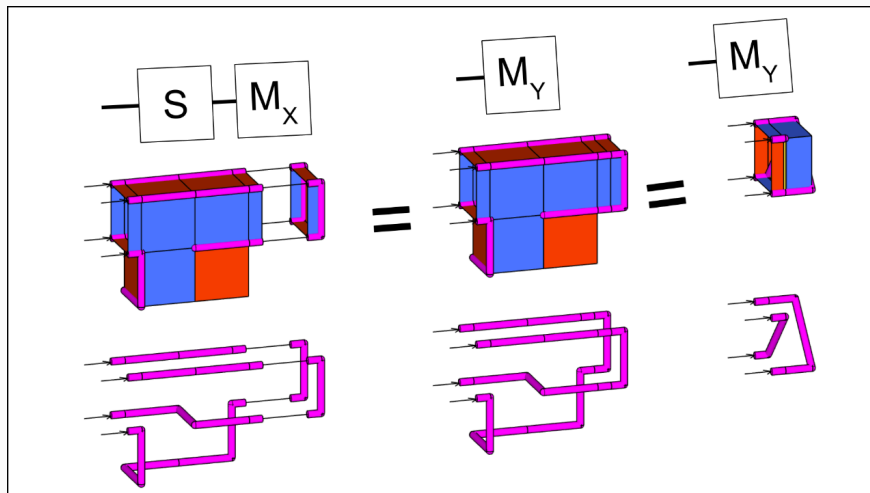
$S$  gate can be implemented fault-tolerantly by moving the twist defects in surface code with code deformation[12].

However, it requires separating the twist defects far enough to avoid decreasing the code distance, which needs heavy resource overhead.

A better way to implement the  $S$  gate is by *gate teleportation*, which requires fault-tolerant  $Y$ -basis initialization or measurement[13].



# Universal Gateset: Y-basis Init/Meas



1. Topologically,  $M_Y$  can be implemented by fusing the pairs of twists along diagonal lines.
2. Diagonal movements of twists can be implemented by ending domain walls in the bulk of the surface code.
3. Domain walls can be implemented by the walking surface code[14] in the bulk.

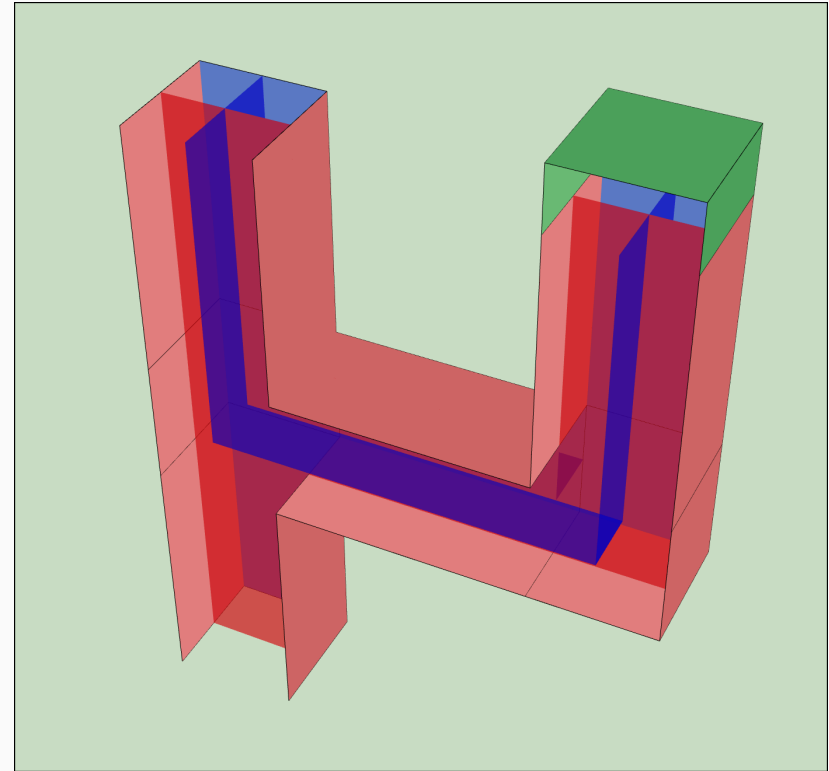
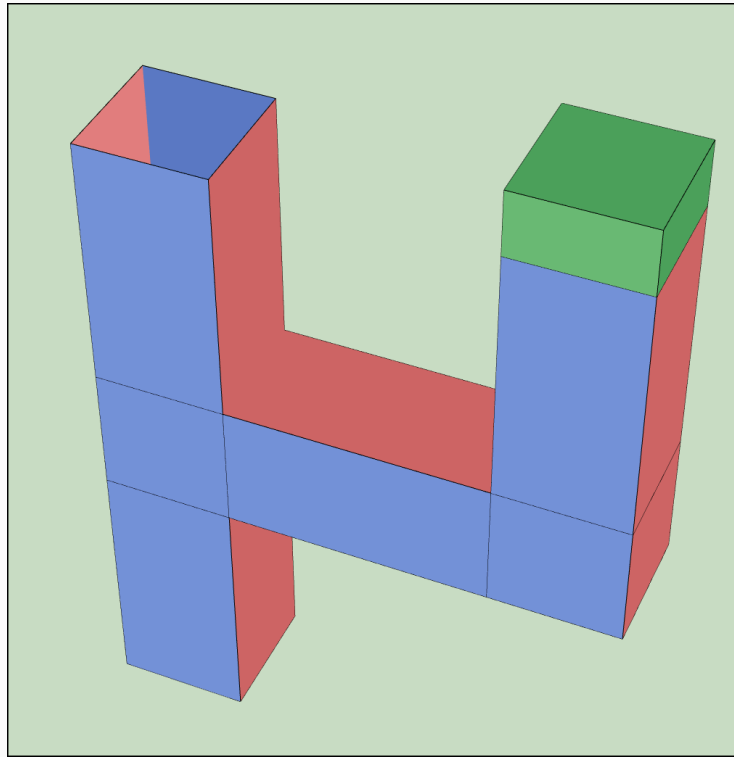
$$O(d^2 \times \frac{d}{2}) \text{ spacetime volume}$$

# Universal Gateset: $S$

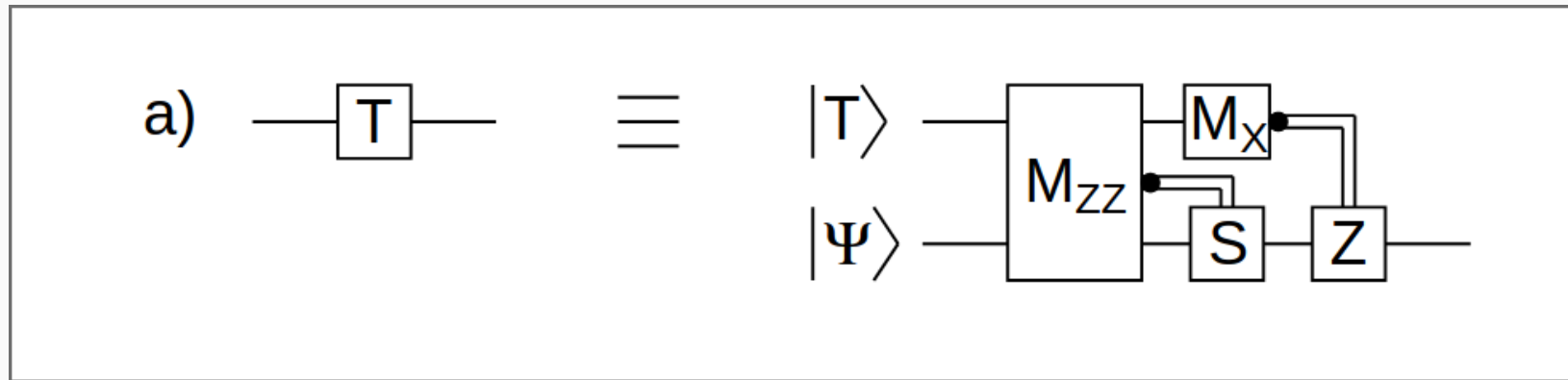
$$Z \xrightarrow{S} Z,$$

$$X \xrightarrow{S} Y$$

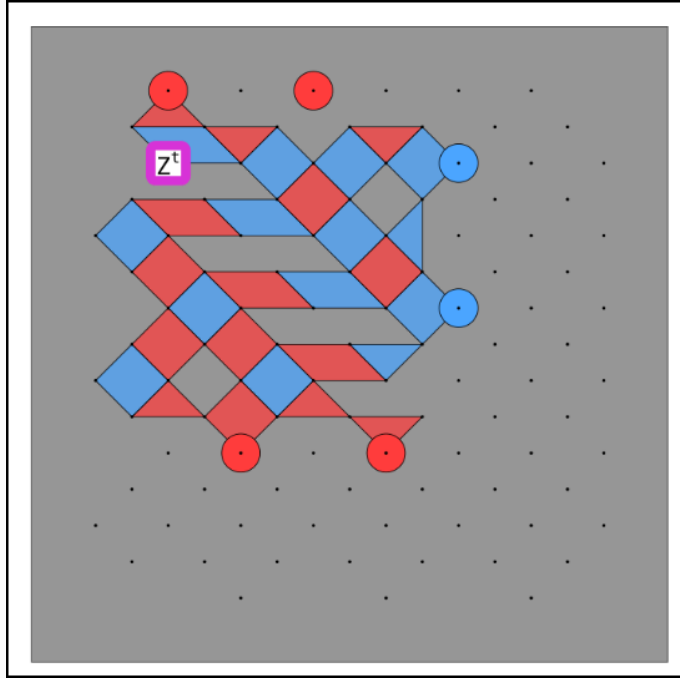
$O(2d^2 \times 1.5d)$  spacetime volume



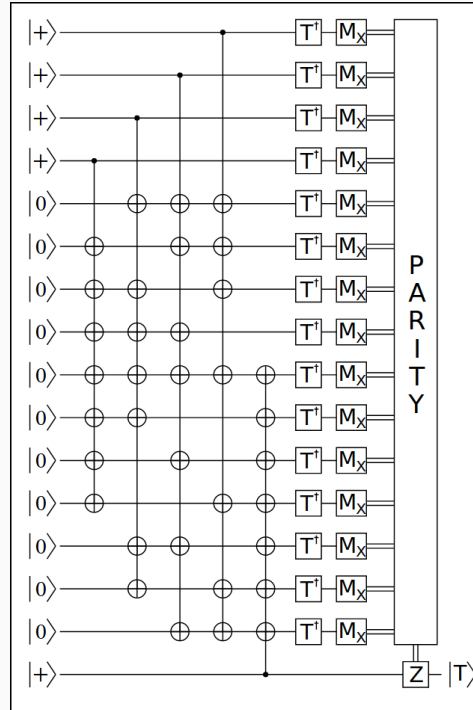
Given a high-fidelity logical  $|T\rangle$  state, we can implement the  $T$  gate by gate teleportation similar to the  $S$  gate:



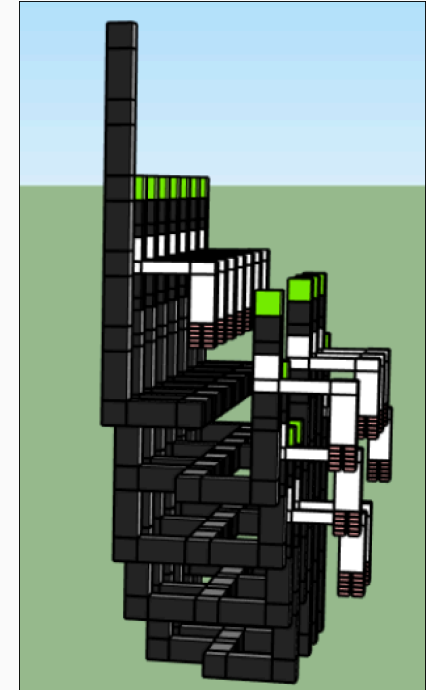
# Magic State Injection and Distillation



Magic State Injection[15]



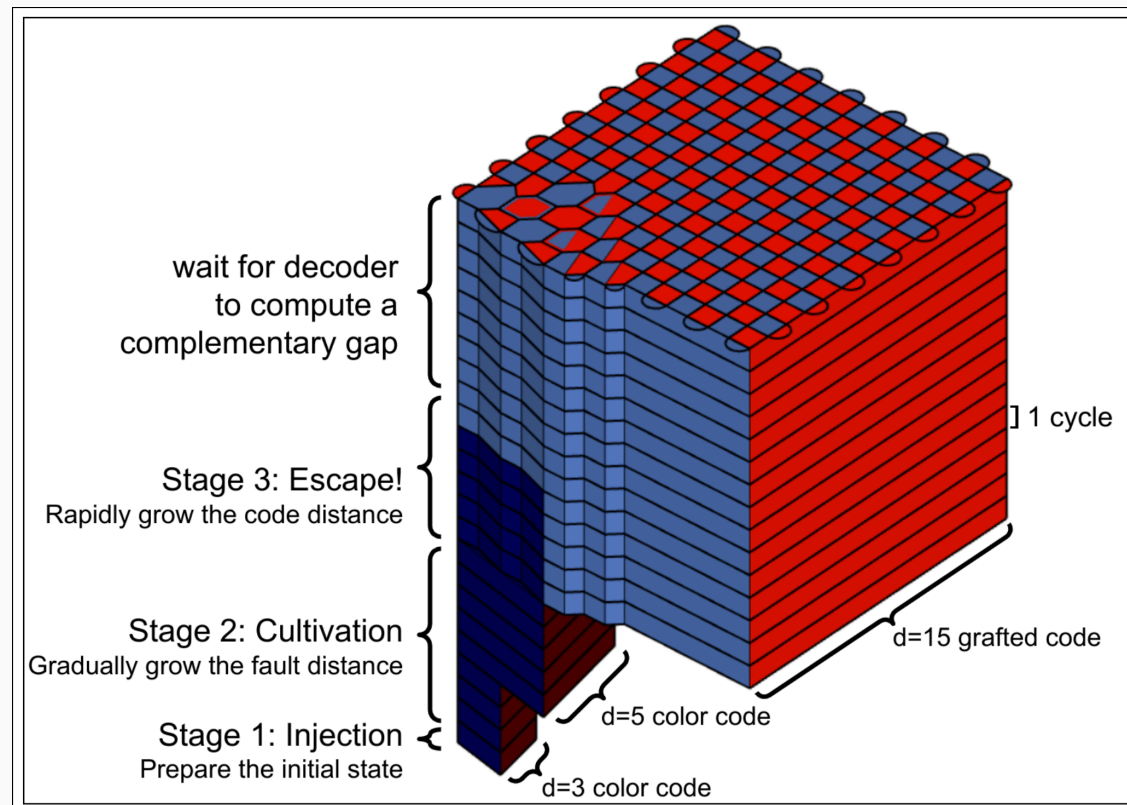
15-1 Distillation



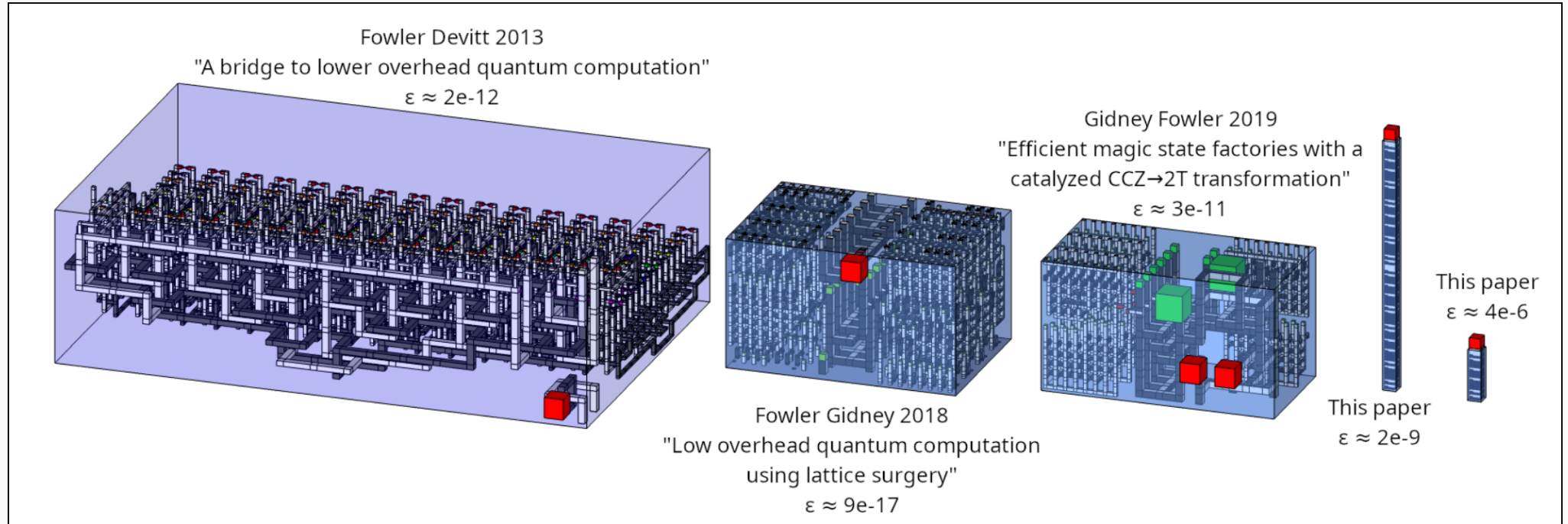
15-1 Distillation in 3D[16]

# Magic State Cultivation

- Color code has the transversal  $H_{XY}$  gate that can be used to check and post-select the  $|T\rangle$  state[17].
- Inject the  $|T\rangle$  state into the color code, grow the fault distance incrementally during checking the logical values.
- Escape the magic state into a “grafted” matchable code and increase the host code distance rapidly.



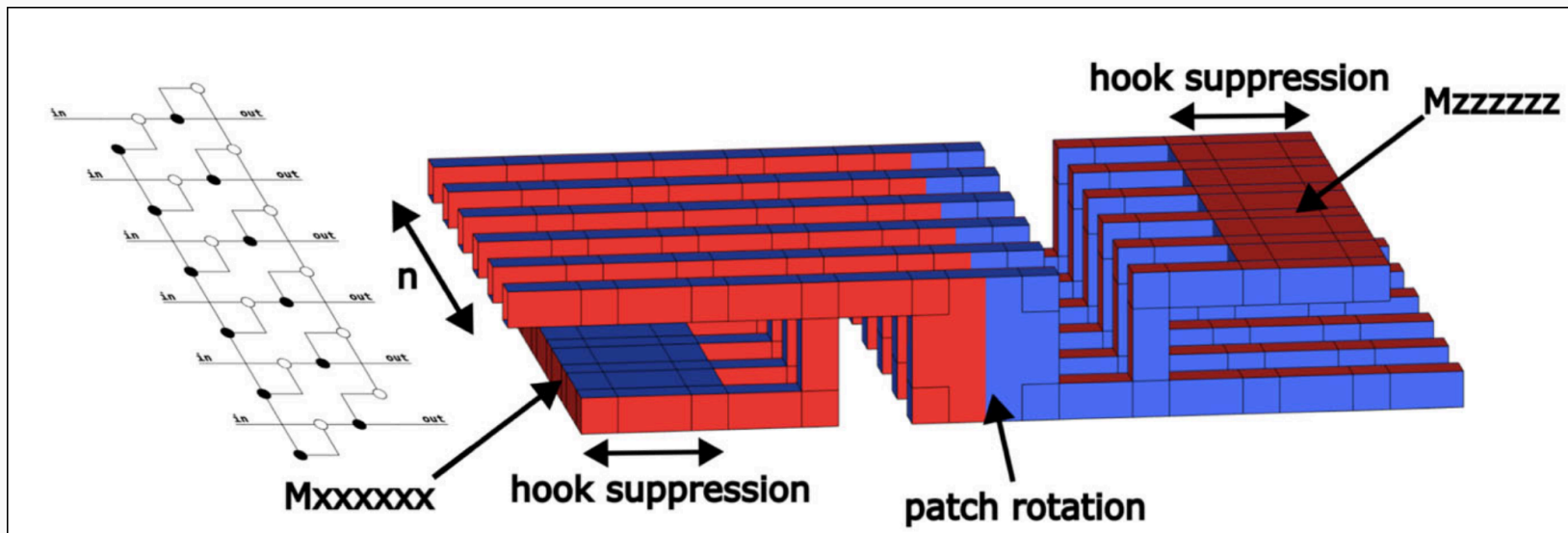
# Magic State Cultivation



# Yoked Surface Code

In large-scale quantum algorithms, the storage of idling logical qubits contributes to a large portion of logical error rate. *Yoked Surface Code* use the simple idea of code concatenation to reduce the overhead of storing logical qubits[18].

For 1D yoked surface code, two parity bits are used to check the X and Z parity of the row. Therefore, it is a  $[[n, n - 2, 2]]$  code.





### **3. Surface Code Compilation**

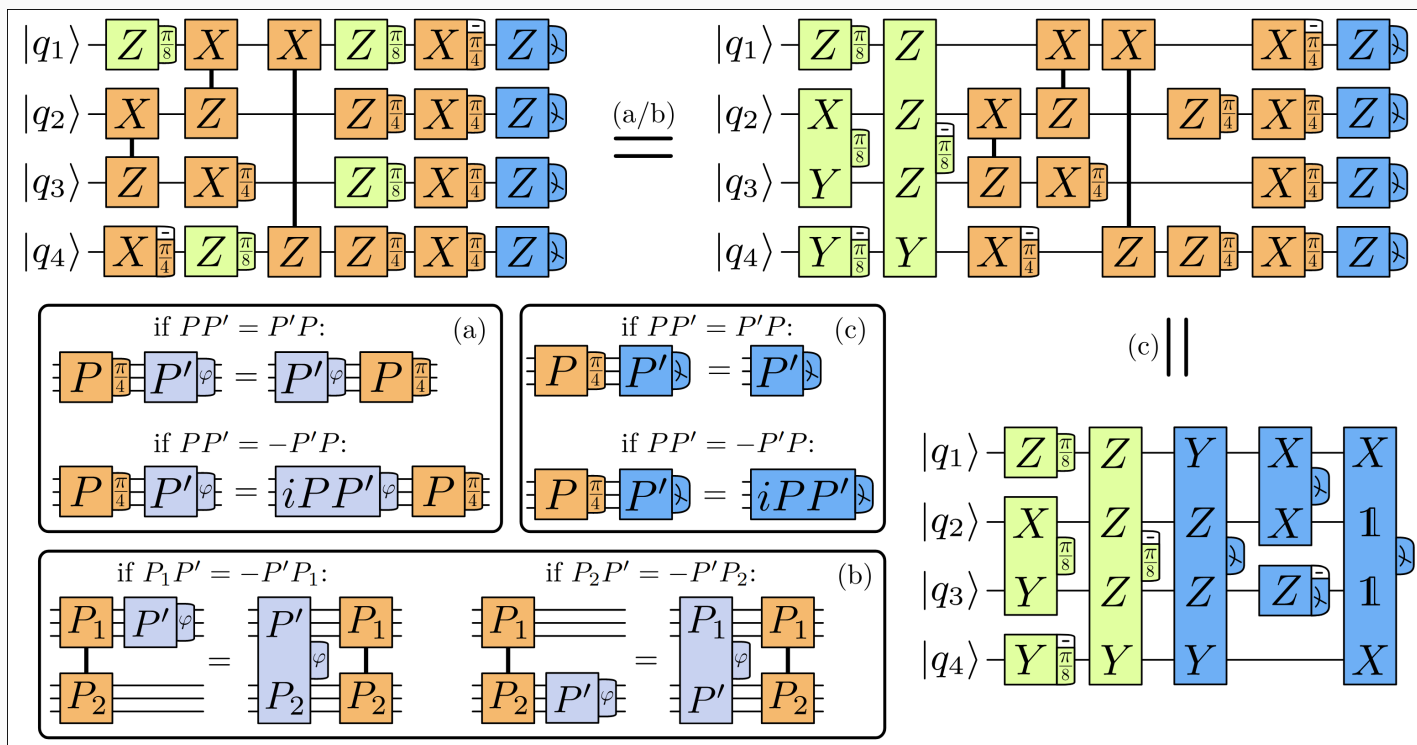
---

Given a quantum circuit that express a quantum algorithm/application, logical circuit compilation need to:

1. Compile it into a circuit/representation consisted of primitive operations that are supported fault-tolerantly by the underlying QECC.
2. Map the components in the compiled circuit/representation onto the coordinate space of the logical qubits.
3. Give a schedule for the serial/parallel execution of the compiled primitive operations.
4. Optimize the compilation with the goal of minimizing the number of logical qubits/algorithm execution time/spacetime volume.

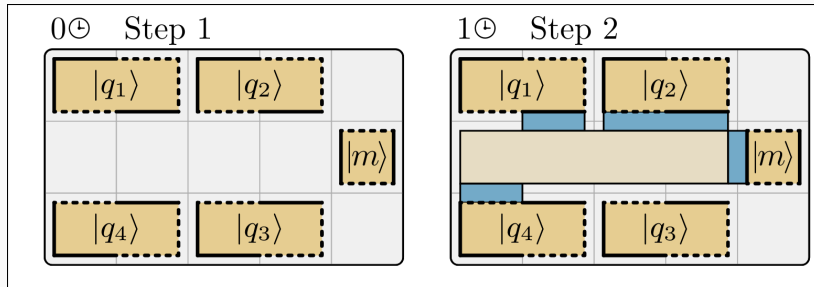
# Sequential Pauli-based Computation

*Sequential Pauli-based Computation*[19] moves all the Clifford gates to the end of the circuit and absorbs them with the final measurements. After that, the circuit consists of solely joint Pauli measurements and  $\frac{\pi}{8}$  multi-Pauli rotations.

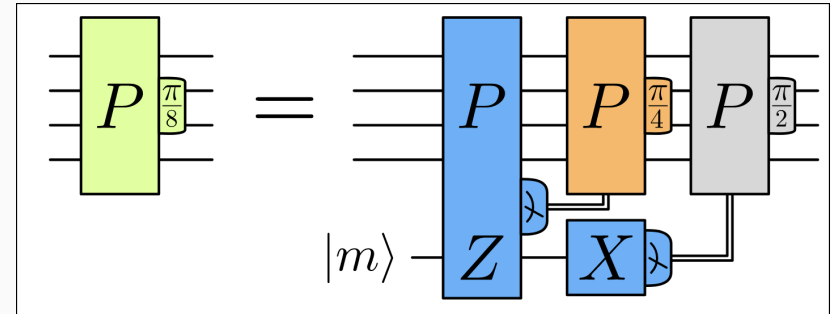


# Sequential Pauli-based Computation

## Primitive operations

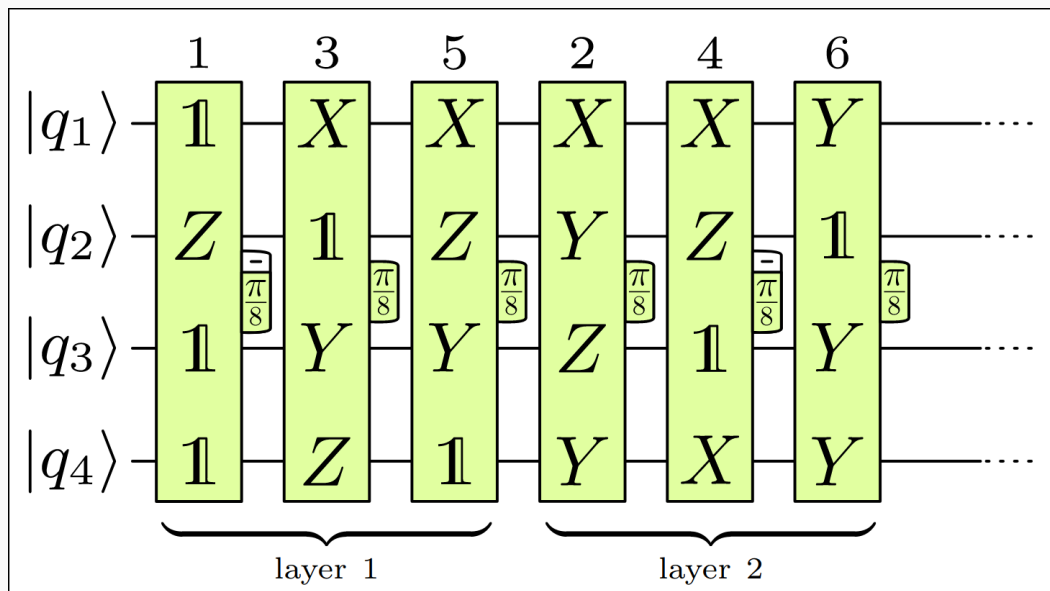


Pauli Product Measurement

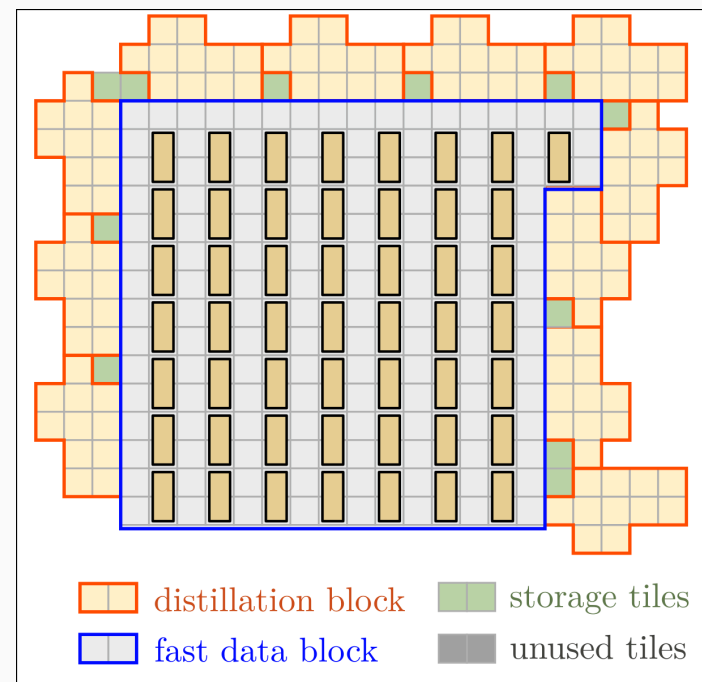


$\frac{\pi}{8}$  Pauli Rotation by consuming a magic state

# Sequential Pauli-based Computation



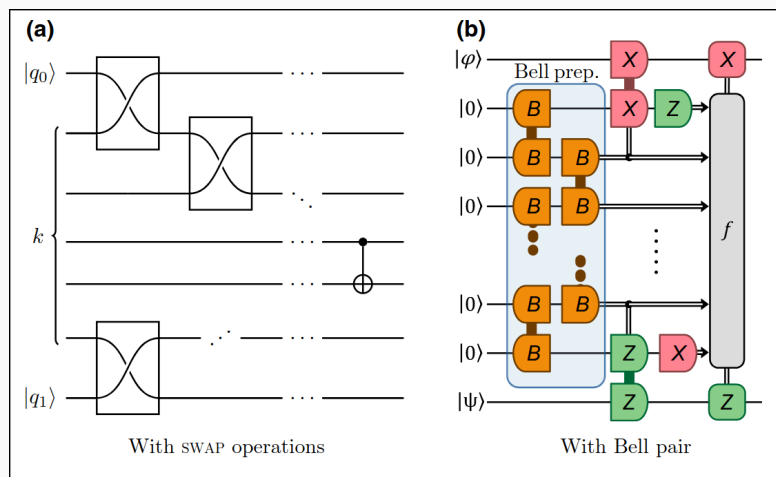
Rewrite circuit as consecutive  $\frac{\pi}{8}$  rotations



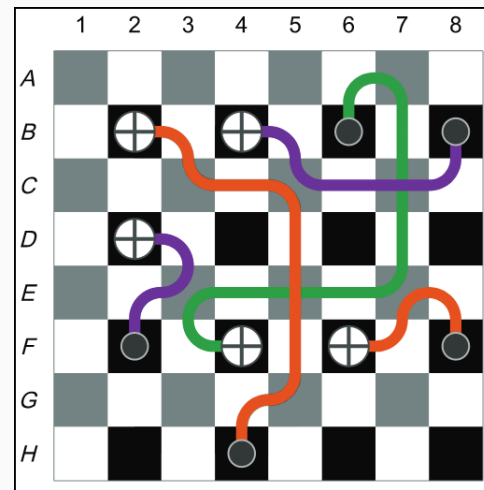
Block Layout

# Clifford + T Compilation

Compile a circuit into gateset  $\{\text{Init}_Z, \text{Init}_X, M_Z, M_X, \text{CNOT}, X, Y, Z, H, S, T\}$  and mapping the gates onto the data blocks with path-finding algorithms to increase the parallelism of the circuit execution while maintaining low-weight parity measurements[20].



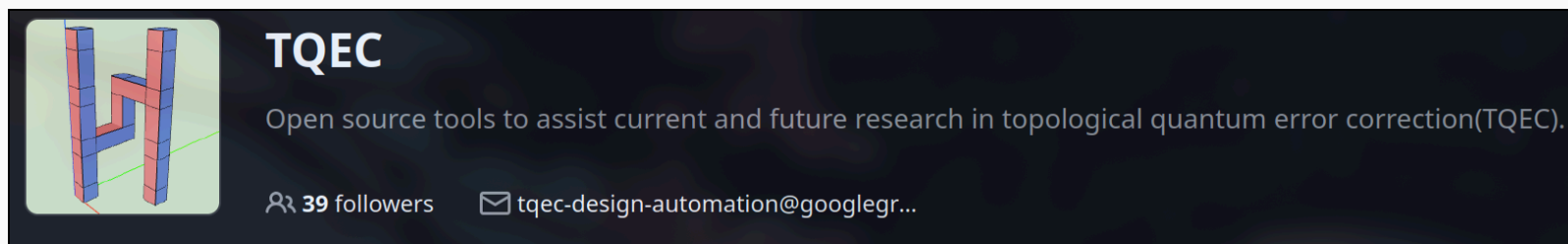
Long-range CNOTs with Bell Pairs and parity measurements



Routing parallelizable CNOTs with vertex-disjoint paths(VDP) and edge-disjoint paths(EDP).

*TQEC* is a project with open community collaboration to develop open-source tools for topological quantum error correction. Currently, it mainly focuses on the representation and compilation of 2D planar surface codes.

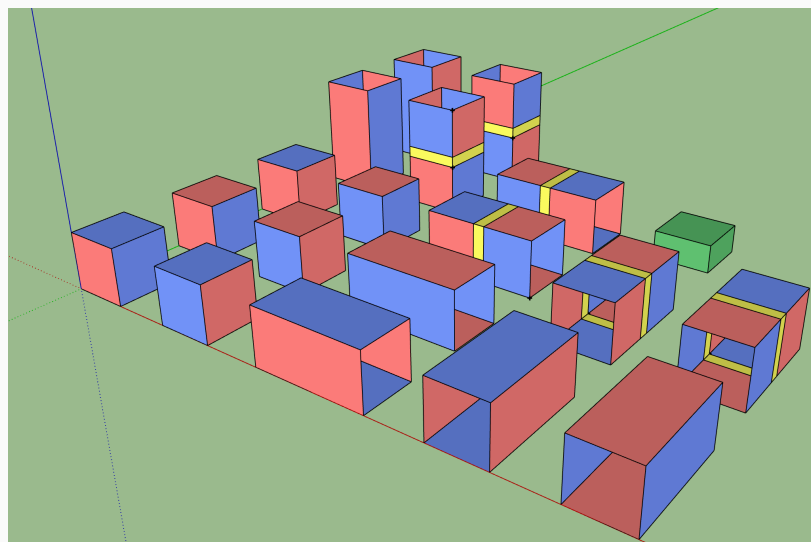
The project is initially organized and led by Dr. Austin Fowler.



Austin Fowler

# Block Diagram Representation

A quantum computation is represented by composing the following building blocks (not complete). Each block has a carefully designed physical circuit associated with it.

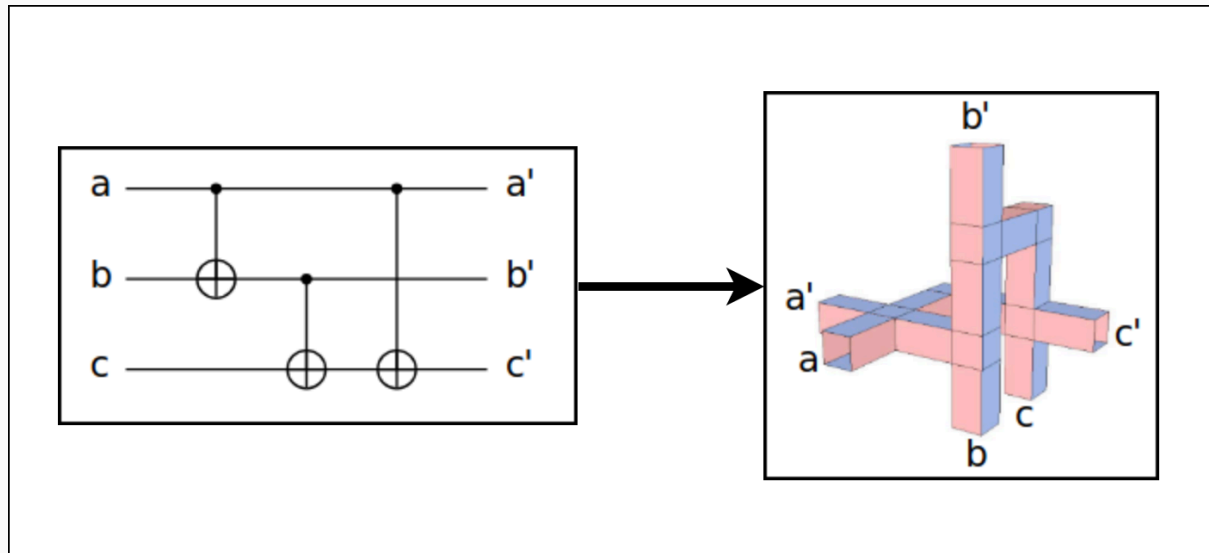


The project focuses on two levels of compilation mediated by the block diagram representation.



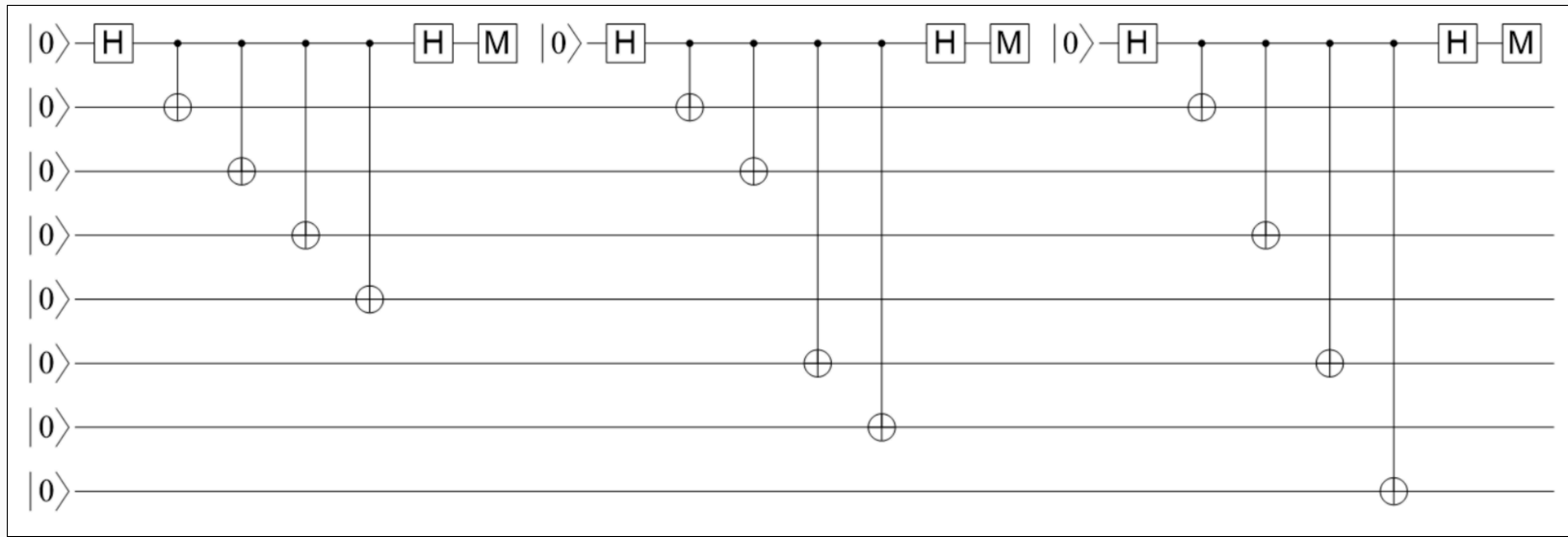
# Logical-level Compilation

*Logical-level Compilation:* compile a high-level/intermediate representation of a quantum circuit, e.g. circuit/ZX diagram, into a low-level representation that specifies all the details needed to layout the circuit onto the hardware (2+1D).



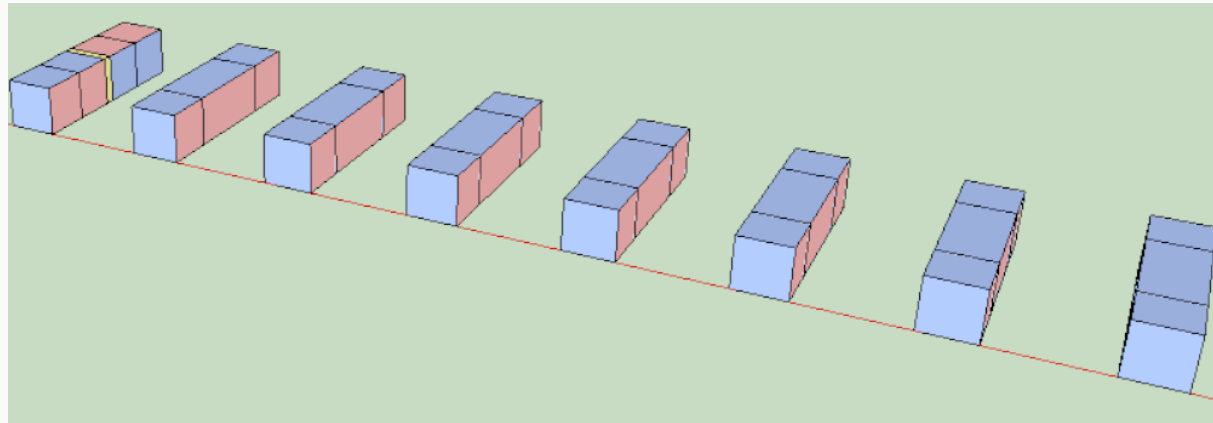
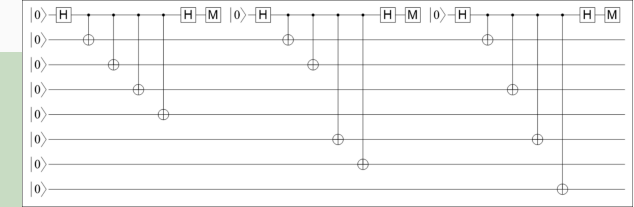
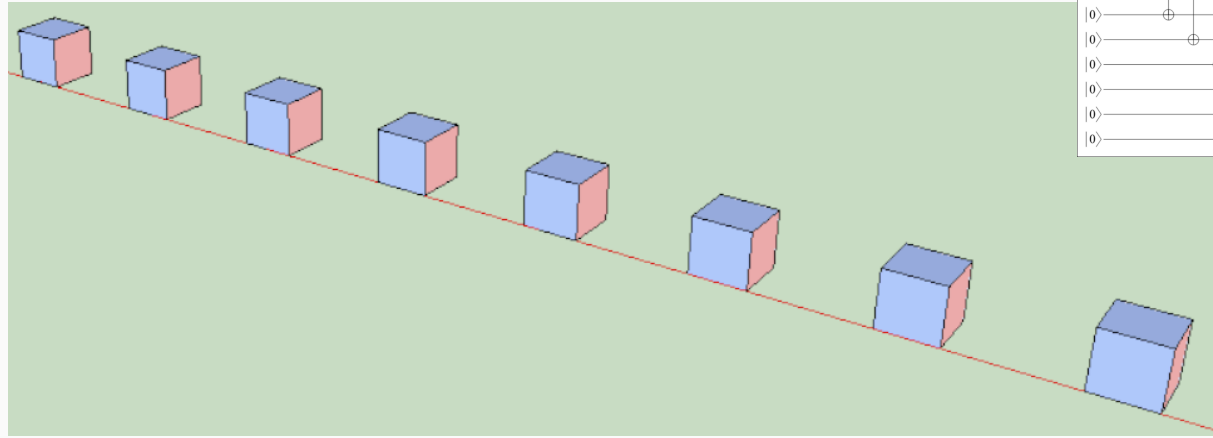
# Logical Compilation by Hand

Consider compiling a following Steane-encoding circuit:

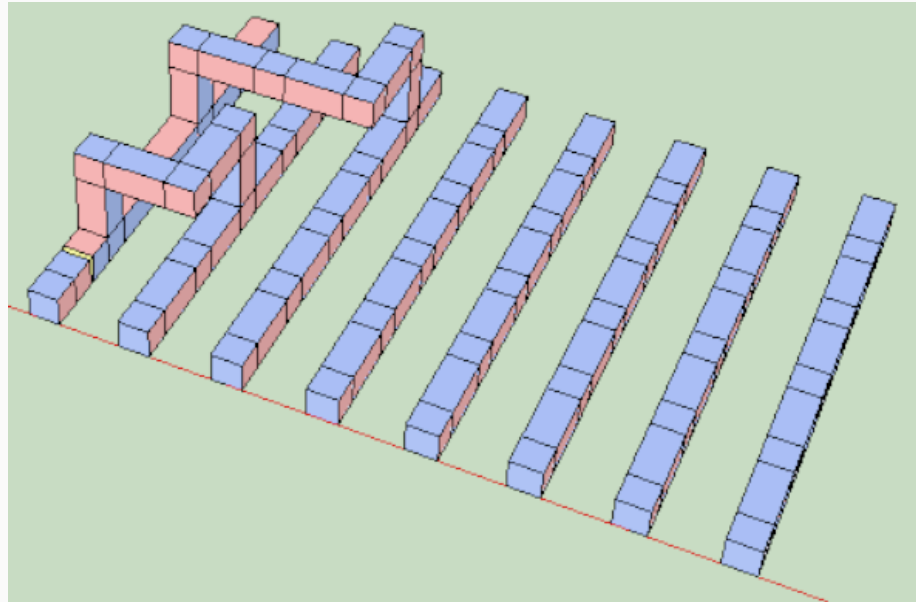
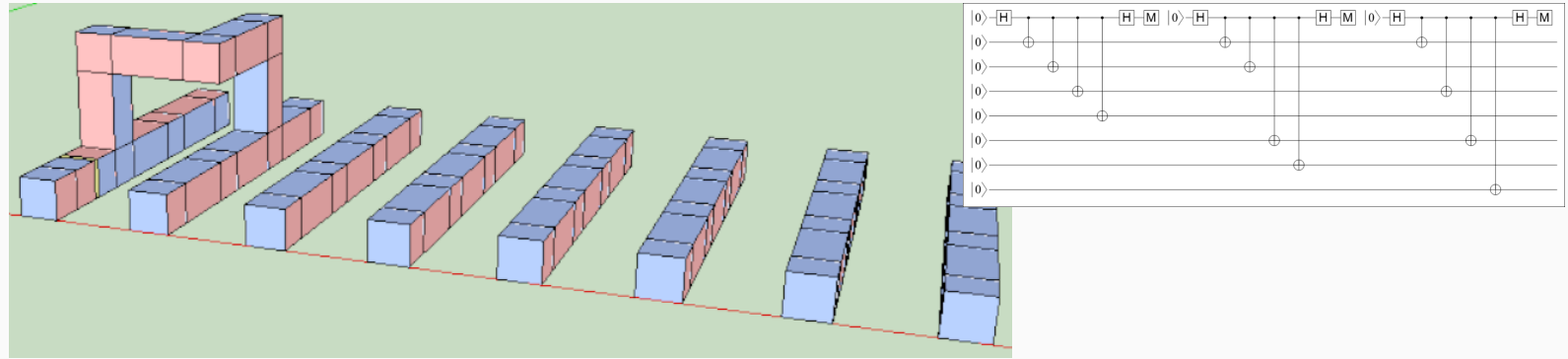


We can build it directly in 3D modeling software like SketchUp.

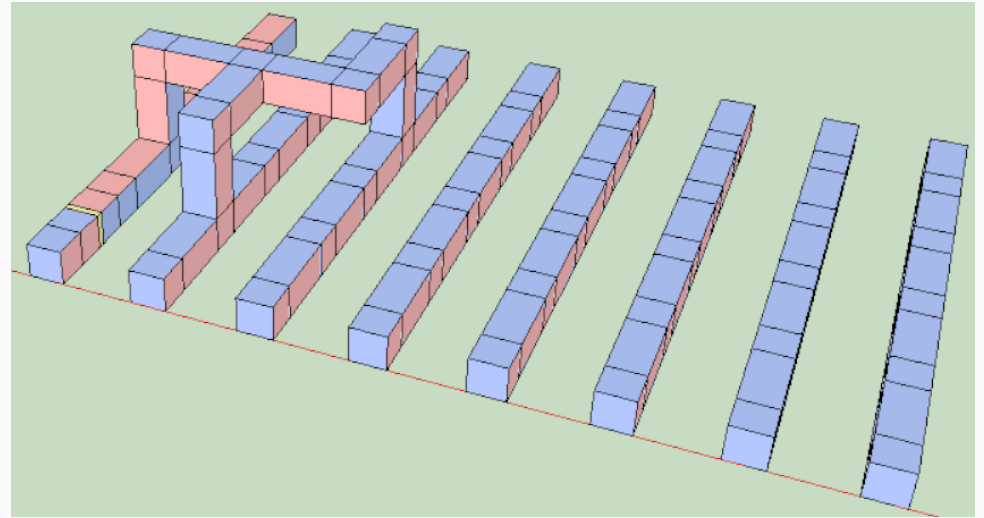
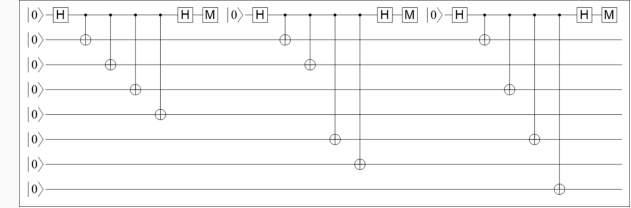
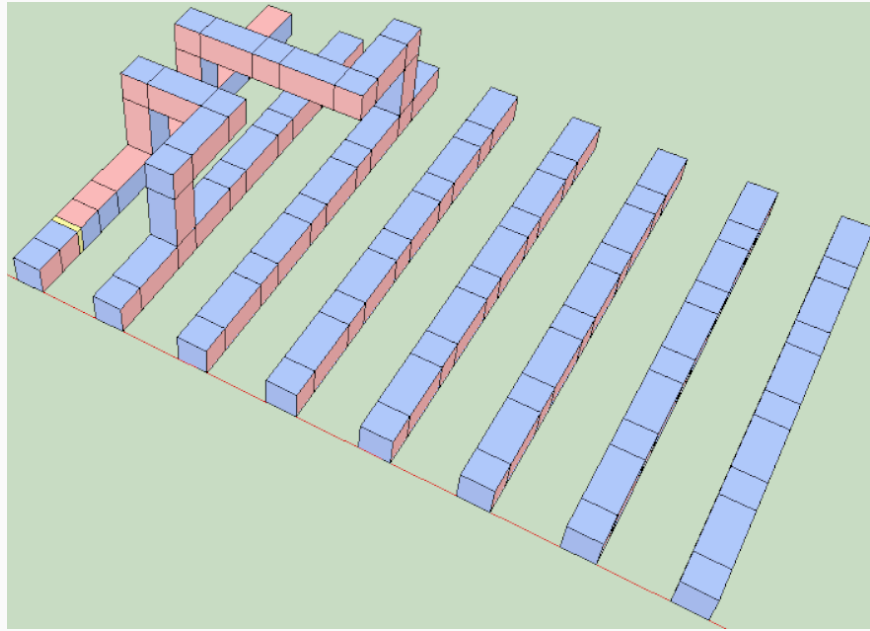
# Logical Compilation by Hand



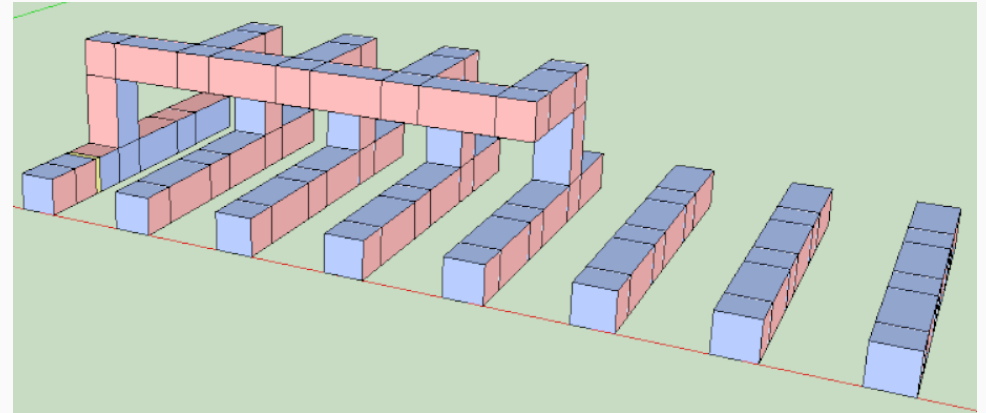
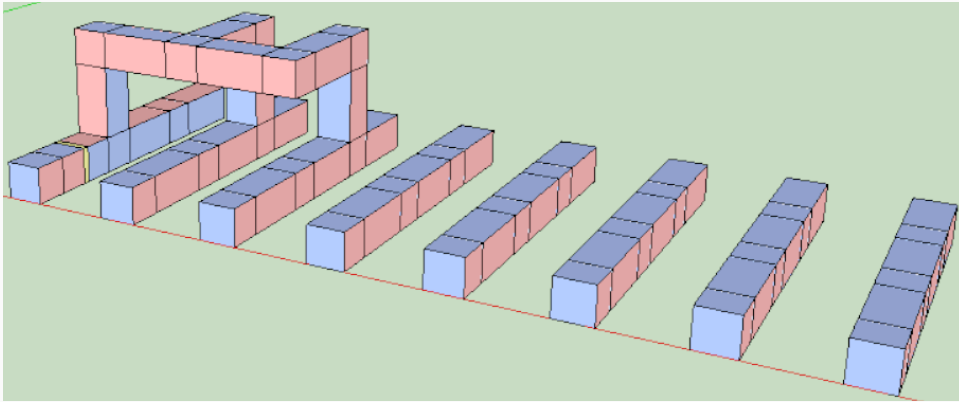
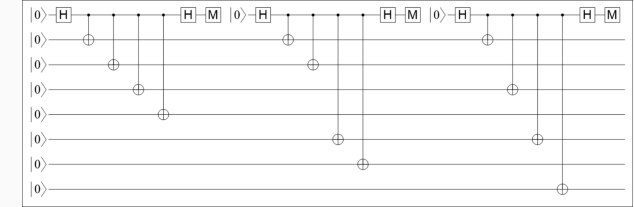
# Logical Compilation by Hand



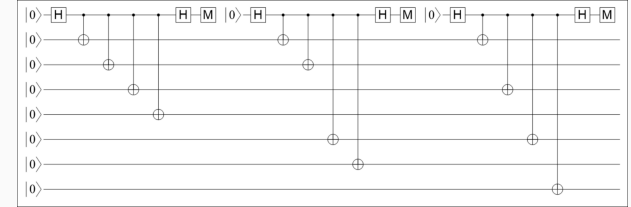
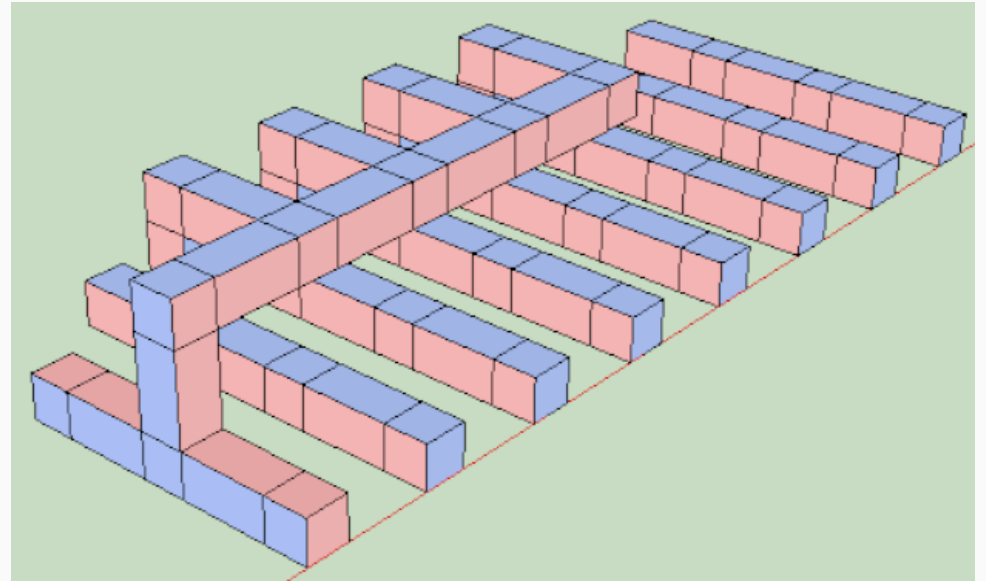
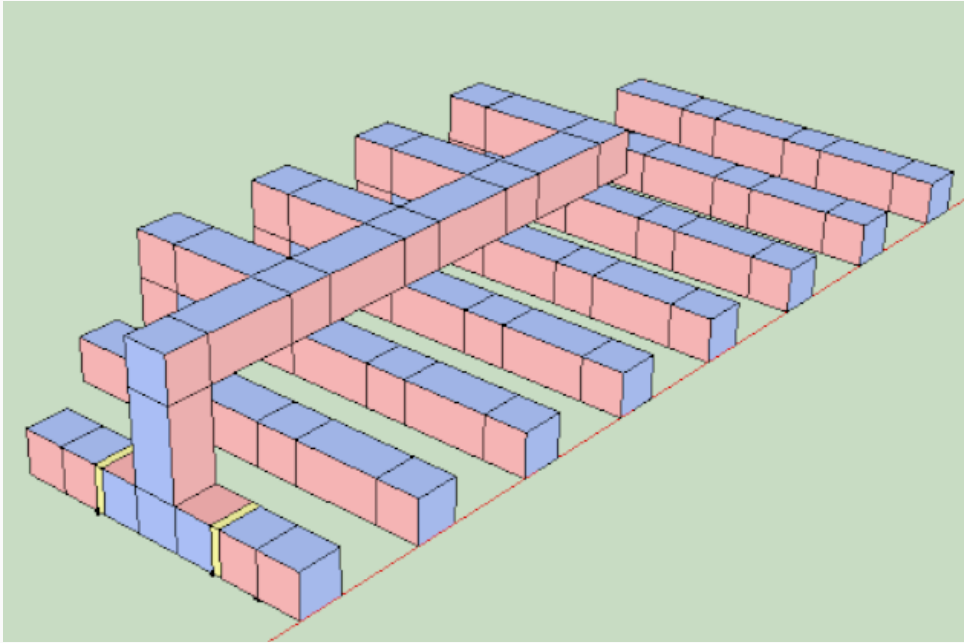
# Logical Compilation by Hand



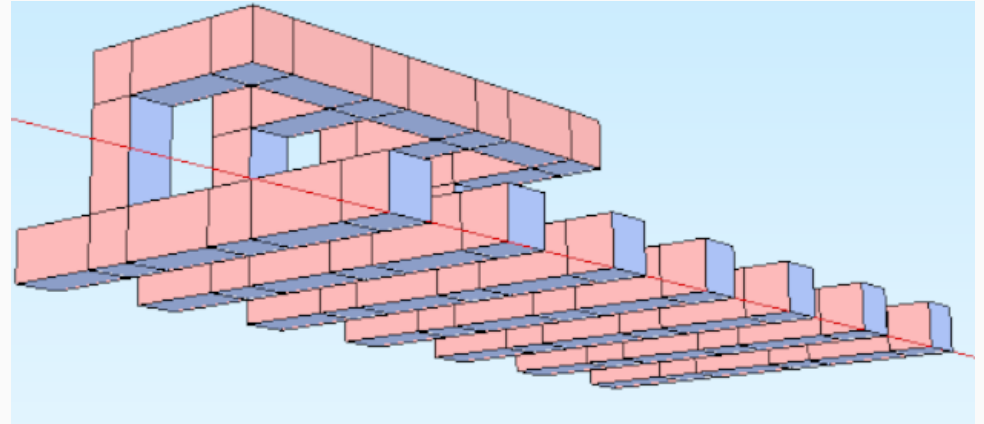
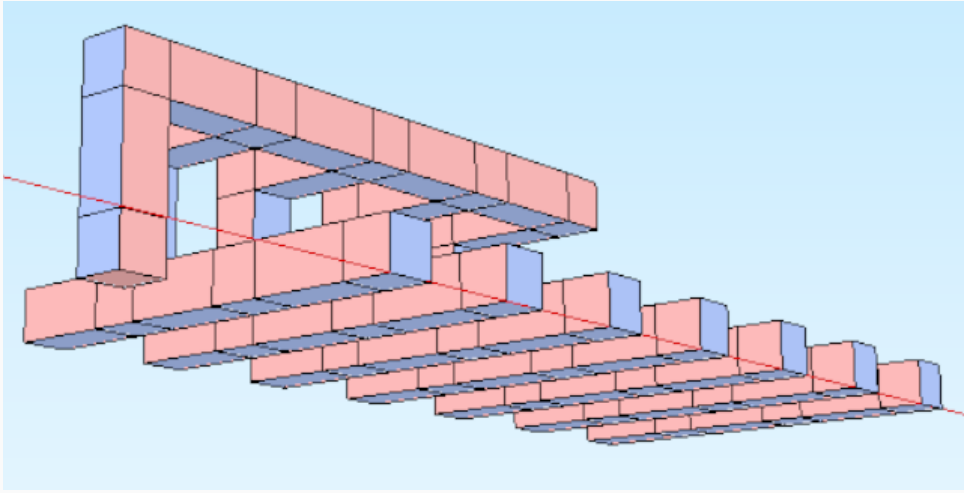
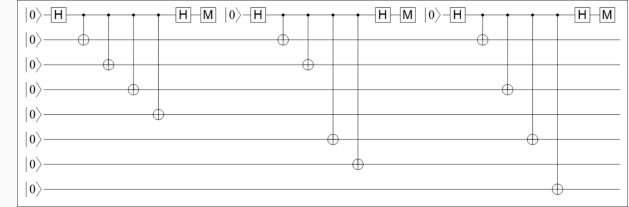
# Logical Compilation by Hand



# Logical Compilation by Hand

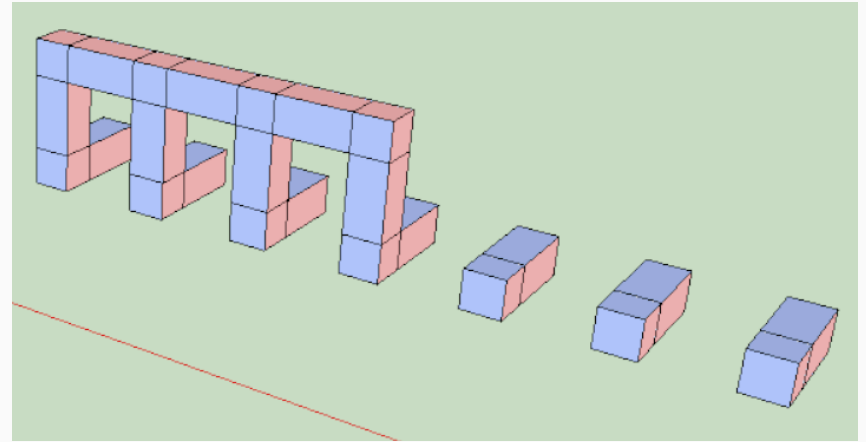
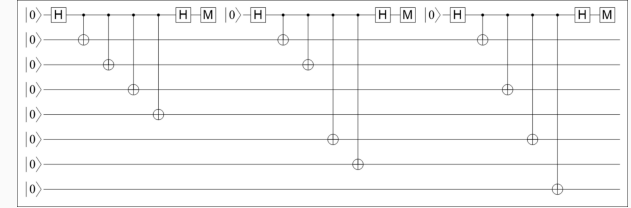
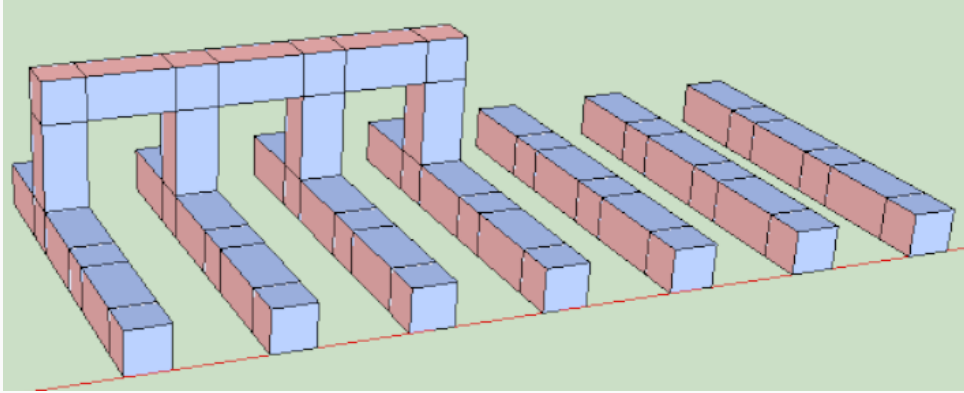


# Logical Compilation by Hand

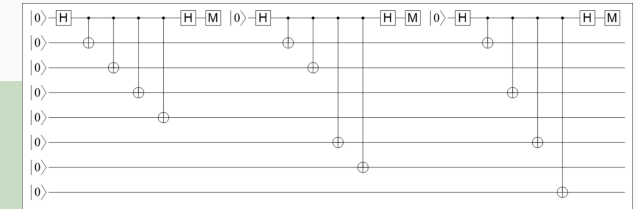
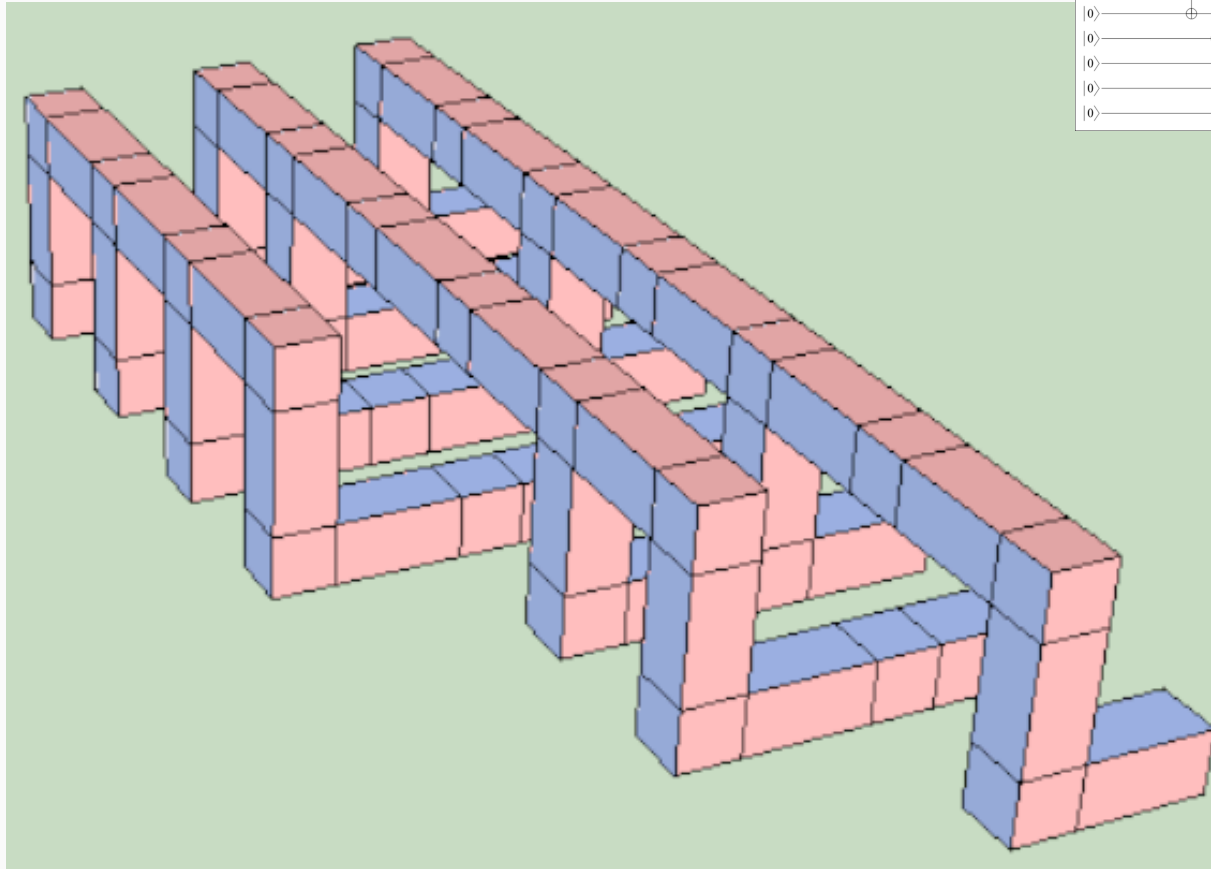




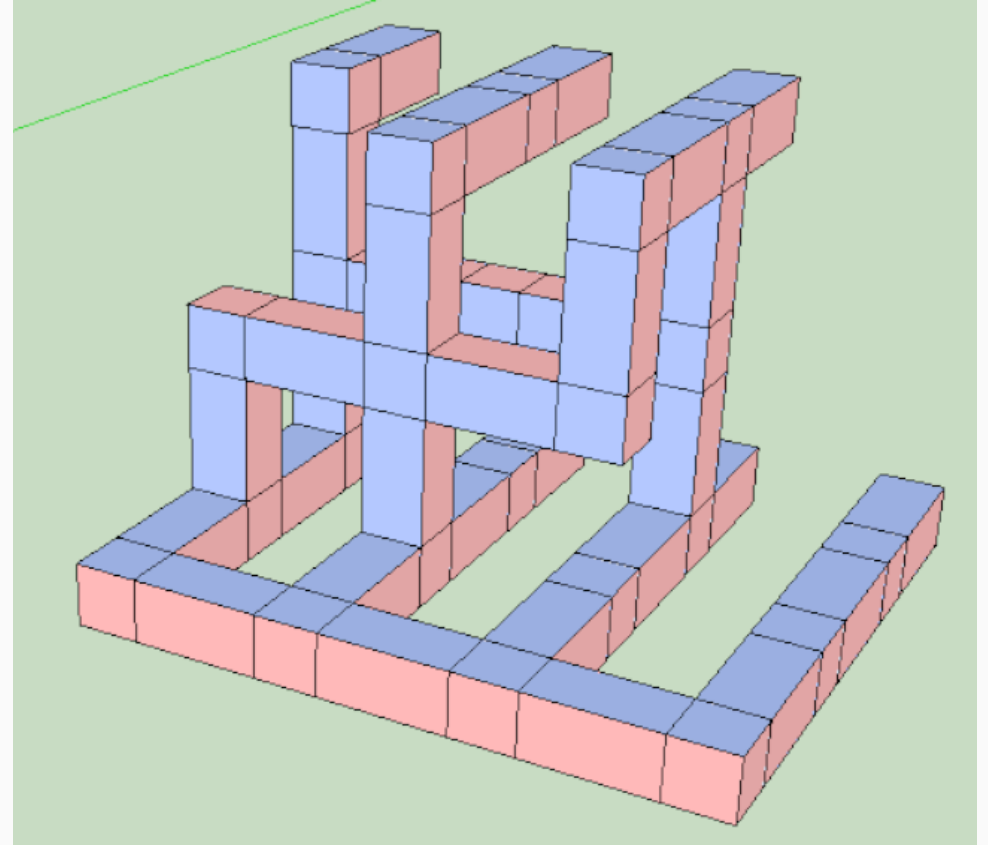
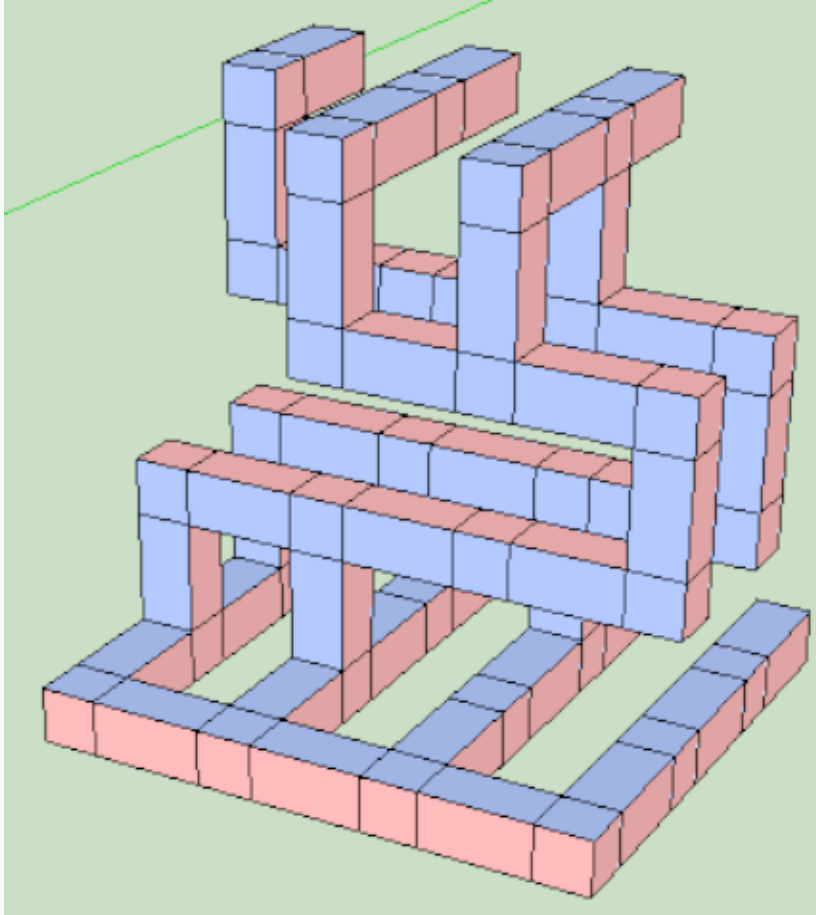
# Logical Compilation by Hand



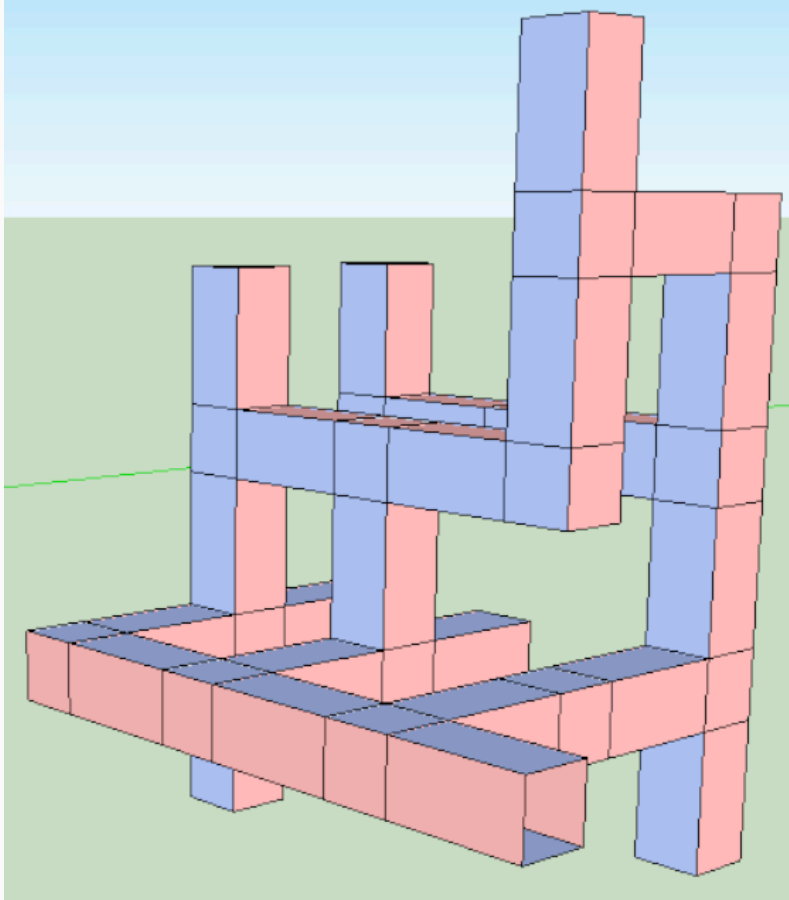
# Logical Compilation by Hand



# Logical Compilation by Hand



# Logical Compilation by Hand

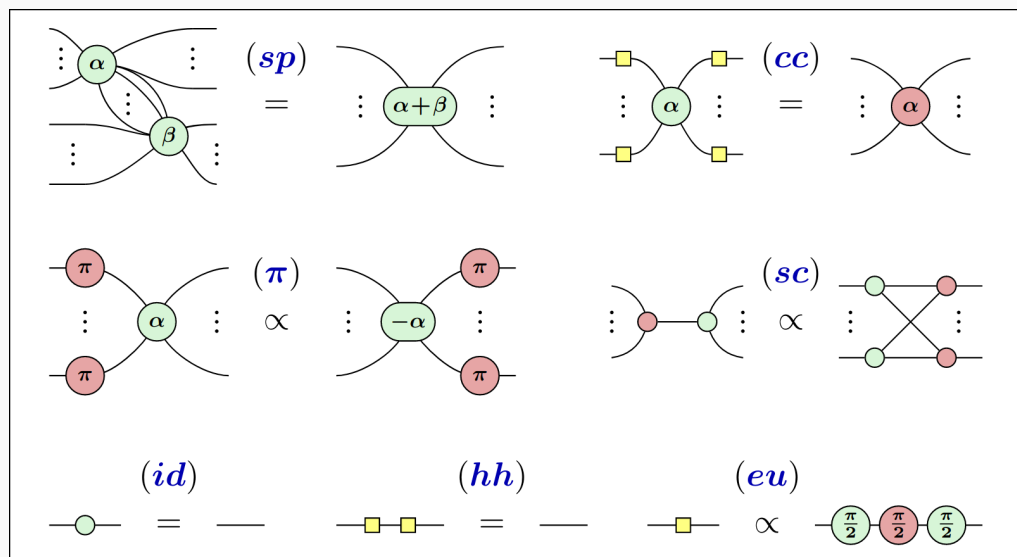


- compiled spacetime volume: 16
- spacetime symmetry in this computation
- all circuit element like CNOT, H, Initialization, Measurement is deformed into blocks.
- inputs/outputs can be moved to connect to the rest of the algorithm.

# ZX-Calculus

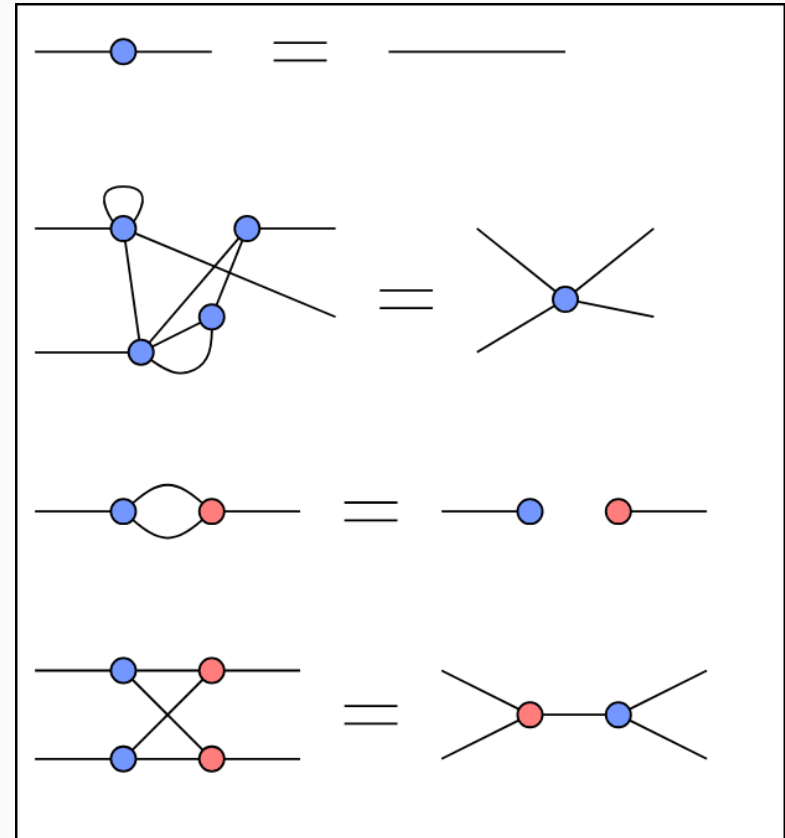
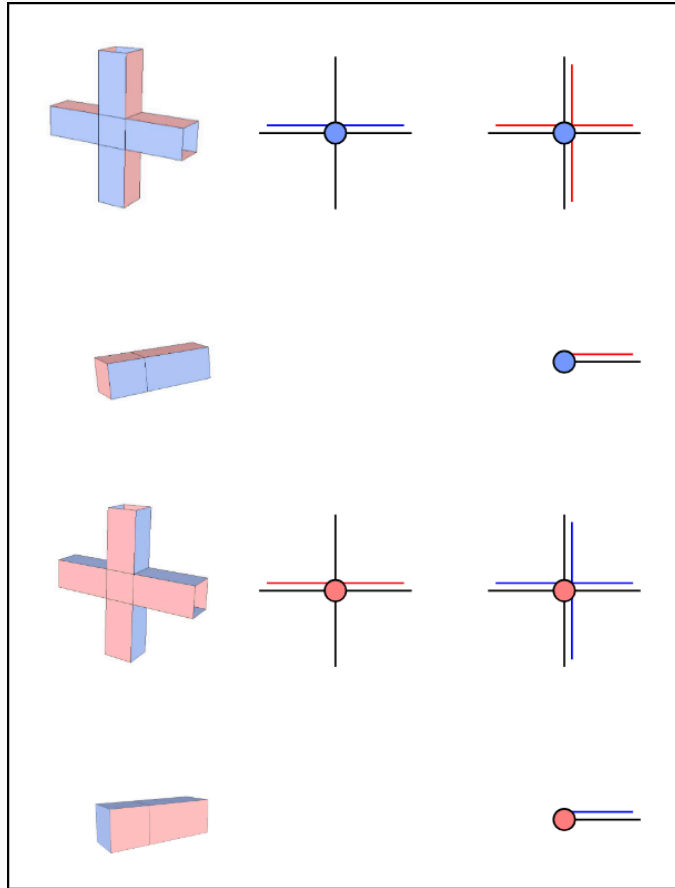
The *ZX-calculus*[21] is a graphical language that goes beyond circuit diagrams. It splits the atom of well-known quantum logic gates to reveal the compositional structure inside.

| Z-spiders                                                                                                                                                                                                                                               | X-spiders                                                                                                                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\begin{array}{c} \vdots \\ \vdots \end{array} \begin{array}{c} \diagup \diagdown \\ \alpha \end{array} \begin{array}{c} \vdots \\ \vdots \end{array} = \\  0 \cdots 0\rangle \langle 0 \cdots 0  + e^{i\alpha}  1 \cdots 1\rangle \langle 1 \cdots 1 $ | $\begin{array}{c} \vdots \\ \vdots \end{array} \begin{array}{c} \diagup \diagdown \\ \alpha \end{array} \begin{array}{c} \vdots \\ \vdots \end{array} =  +\cdots+\rangle \langle +\cdots+  + \\ e^{i\alpha}  -\cdots-\rangle \langle -\cdots- $ |

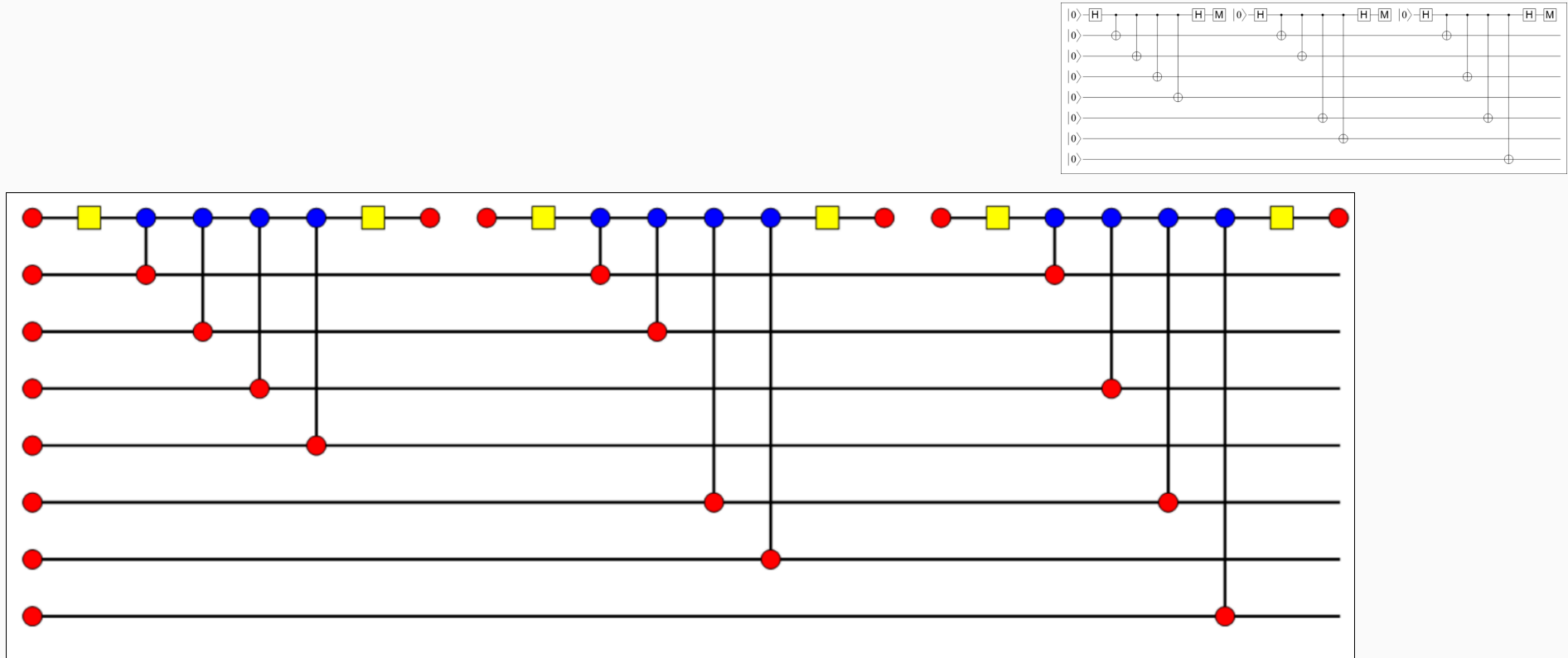


# ZX-Calculus

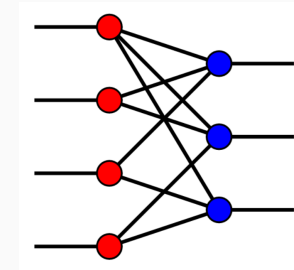
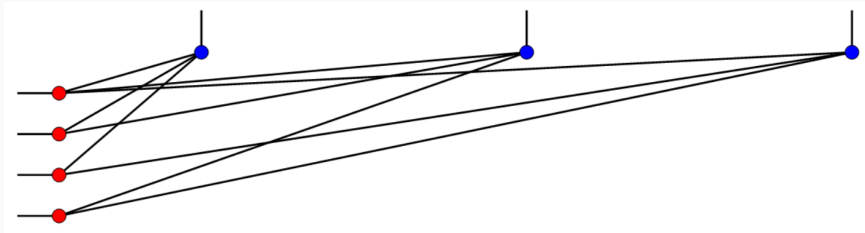
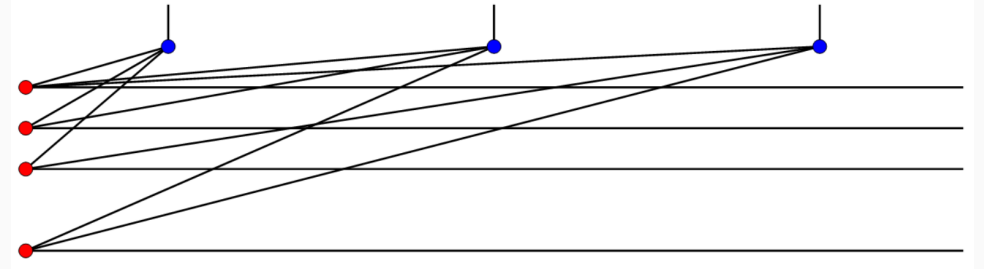
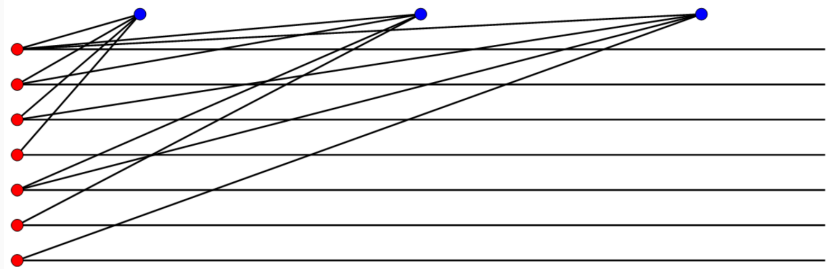
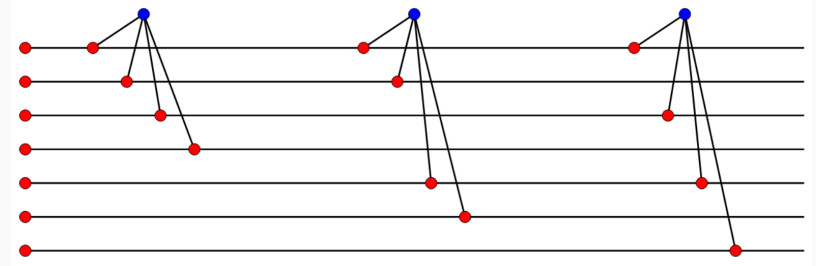
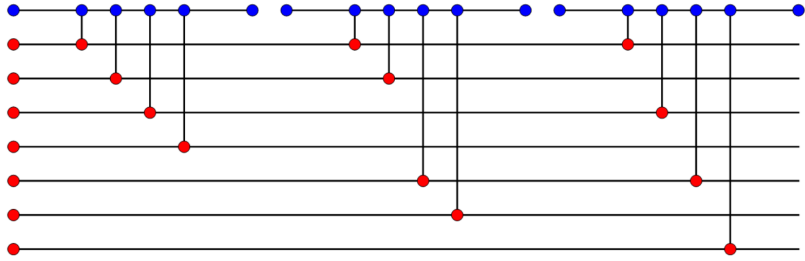
ZX-calculus can be a language of surface code lattice surgery.



## Compile with ZX-Calculus

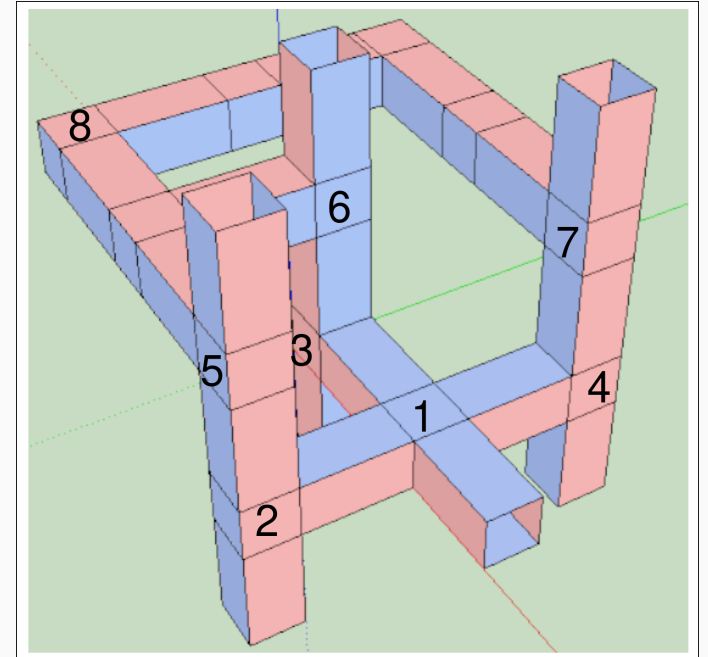
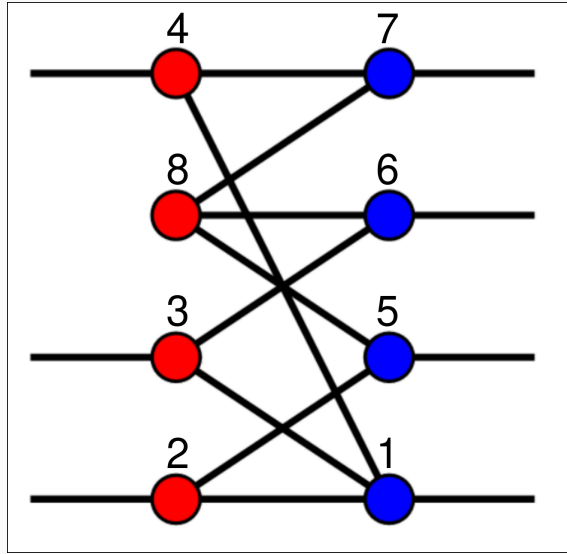
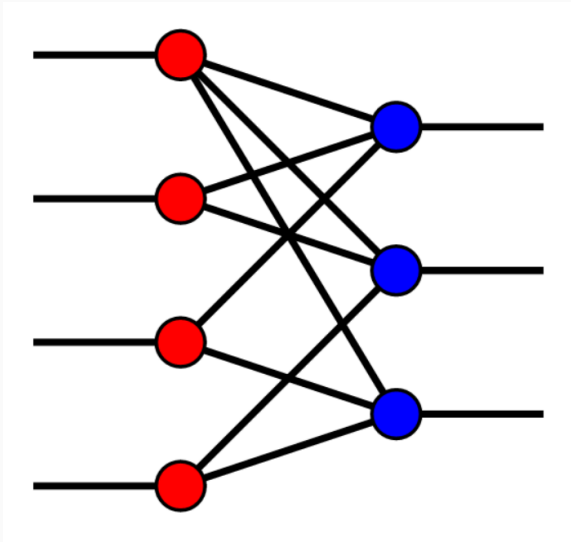


# Compile with ZX-Calculus





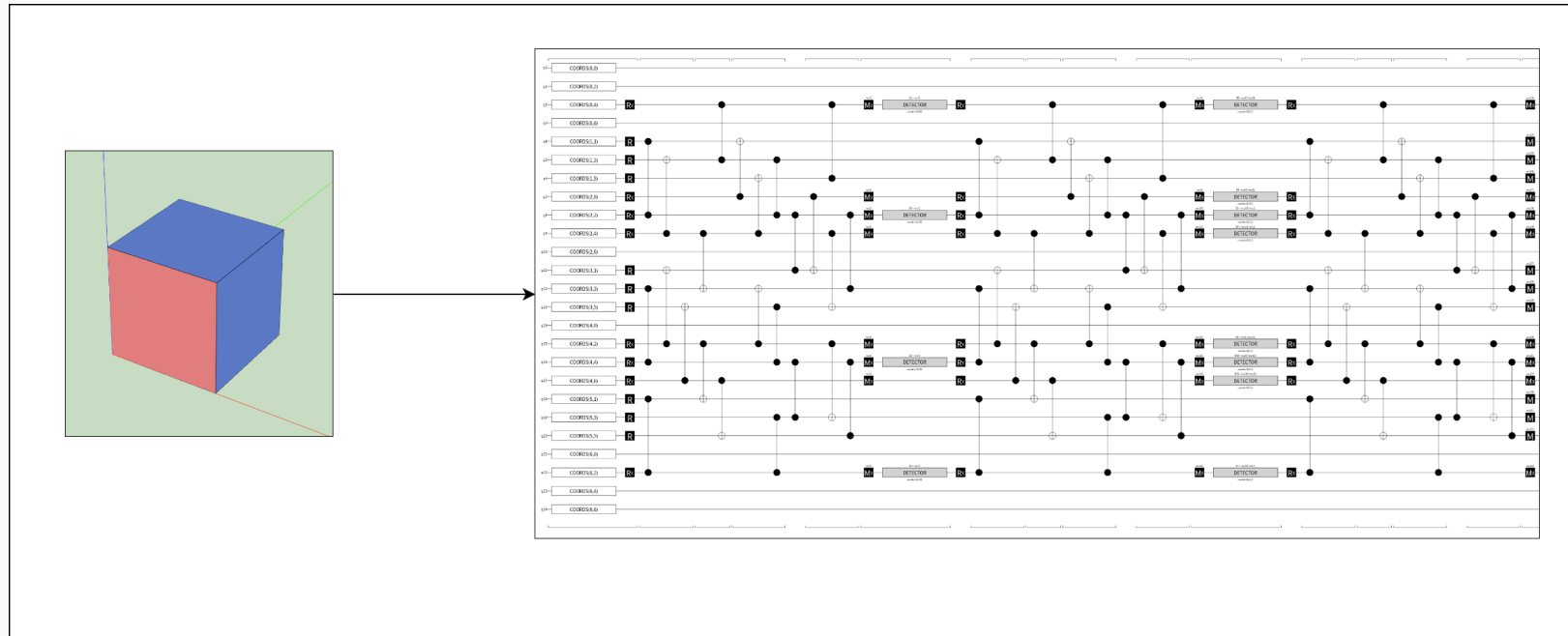
# Compile with ZX-Calculus



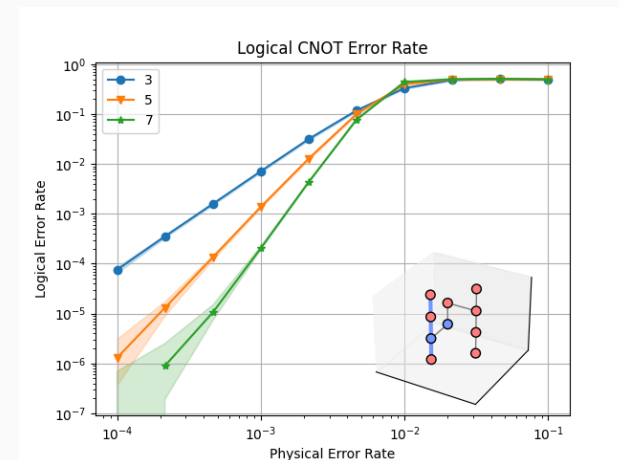
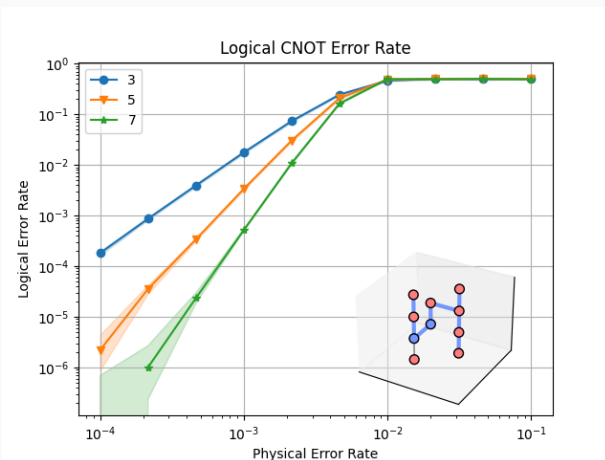
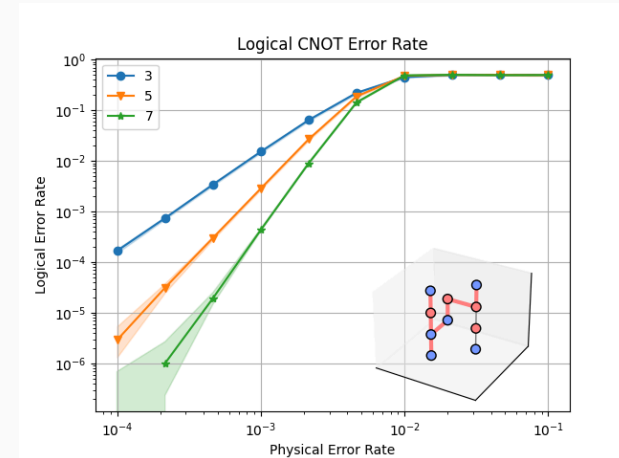
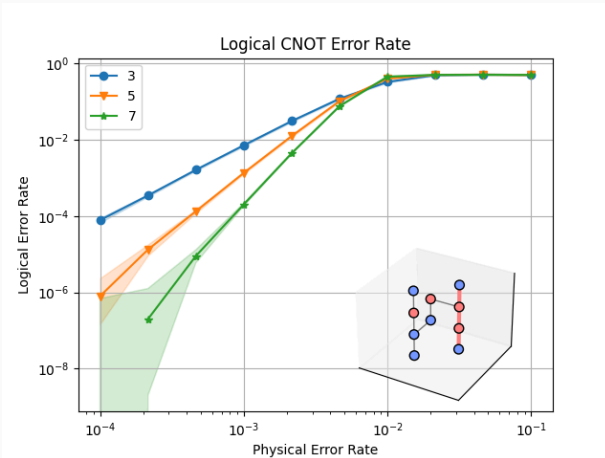
# Physical-level Compilation

*Physical-level Compilation:* compile the low-level representation into a sequence of physical gate instructions that can be executed directly on the hardware.

Physical-level compilation takes care of composing the circuit blocks to generate a fault-tolerant circuit annotated with correct detectors and observables.



# Physical-level Compilation



# Current Status and Future Work

## Current Status:

- All Clifford block circuits except Y-basis initialization and measurement.
- Algorithm combined with SAT solver for circuit detectors automation.
- Algorithm for finding the minimal generating set of correlation surfaces in the Clifford computation.
- Can successfully compile the Steane-encoding block diagram above into physical circuit that can be simulated with `stim`[22].
- A greedy graph algorithm for compiling a ZX-diagram into a valid block diagram.

## Future Works:

- Support Y-basis initialization and measurement and magic state cultivation blocks.
- More general algorithm for finding the correlation surfaces in universal quantum circuits and returning time ordering of the non-Clifford gates.
- More efficient algorithm for logical compilation. Explore alternative intermediate representations like Quon[23].

Thanks for your attention!

The slides are available online:

[https://github.com/inmzhang/surface\\_code\\_quantum\\_computation/tree/talk/ymsc](https://github.com/inmzhang/surface_code_quantum_computation/tree/talk/ymsc)

## 4. Appendix

---

# Bibliography

- [1] A. M. Dalzell *et al.*, “Quantum Algorithms: A Survey of Applications and End-to-End Complexities,” no. arXiv:2310.03011. arXiv, 2023.
- [2] D. Gao *et al.*, “Establishing a new benchmark in quantum computational advantage with 105-qubit Zuchongzhi 3.0 processor,” *Physical Review Letters*, vol. 134, no. 9, p. 90601, 2025.
- [3] Y. Wu *et al.*, “Strong Quantum Computational Advantage Using a Superconducting Quantum Processor,” *Physical Review Letters*, vol. 127, no. 18, p. 180501, 2021, doi: [10.1103/PhysRevLett.127.180501](https://doi.org/10.1103/PhysRevLett.127.180501).
- [4] M. E. Beverland *et al.*, “Assessing Requirements to Scale to Practical Quantum Advantage,” no. arXiv:2211.07629. arXiv, 2022.
- [5] Riverlane, “Quantum Error Correction Report 2024.” [Online]. Available: <https://www.riverlane.com/quantum-error-correction-report-2024>

- [6] Y. Zhao *et al.*, “Realization of an error-correcting surface code with superconducting qubits,” *Physical Review Letters*, vol. 129, no. 3, p. 30501, 2022.
- [7] Google Quantum AI *et al.*, “Suppressing Quantum Errors by Scaling a Surface Code Logical Qubit,” *Nature*, vol. 614, no. 7949, pp. 676–681, 2023, doi: [10.1038/s41586-022-05434-1](https://doi.org/10.1038/s41586-022-05434-1).
- [8] D. Bluvstein *et al.*, “Logical Quantum Processor Based on Reconfigurable Atom Arrays,” *Nature*, 2023, doi: [10.1038/s41586-023-06927-3](https://doi.org/10.1038/s41586-023-06927-3).
- [9] R. Acharya *et al.*, “Quantum error correction below the surface code threshold,” *arXiv preprint arXiv:2408.13687*, 2024.
- [10] N. Lacroix *et al.*, “Scaling and Logic in the Color Code on a Superconducting Quantum Processor,” no. arXiv:2412.14256. arXiv, 2024. doi: [10.48550/arXiv.2412.14256](https://doi.org/10.48550/arXiv.2412.14256).



# Bibliography

- C. Horsman, A. G. Fowler, S. Devitt, and R. V. Meter, “Surface Code Quantum
- [11] Computing by Lattice Surgery,” *New Journal of Physics*, vol. 14, no. 12, p. 123011, 2012, doi: [10.1088/1367-2630/14/12/123011](https://doi.org/10.1088/1367-2630/14/12/123011).
- B. J. Brown, K. Laubscher, M. S. Kesselring, and J. R. Wootton, “Poking Holes and
- [12] Cutting Corners to Achieve Clifford Gates with the Surface Code,” *Physical Review X*, vol. 7, no. 2, p. 21029, 2017, doi: [10.1103/PhysRevX.7.021029](https://doi.org/10.1103/PhysRevX.7.021029).
- [13] C. Gidney, “Inplace access to the surface code y basis,” *Quantum*, vol. 8, p. 1310, 2024.
- M. McEwen, D. Bacon, and C. Gidney, “Relaxing Hardware Requirements for Surface
- [14] Code Circuits Using Time-dynamics,” *Quantum*, vol. 7, p. 1172, 2023, doi: [10.22331/q-2023-11-07-1172](https://doi.org/10.22331/q-2023-11-07-1172).
- [15] C. Gidney, “Cleaner Magic States with Hook Injection,” no. arXiv:2302.12292. arXiv, 2023.

# Bibliography

- [16] C. Gidney and A. G. Fowler, “Efficient Magic State Factories with a Catalyzed  $|\text{CCZ}\rangle$  to  $2|\text{T}\rangle$  Transformation”, *Quantum*, vol. 3, p. 135, 2019, doi: [10.22331/q-2019-04-30-135](https://doi.org/10.22331/q-2019-04-30-135).
- [17] C. Gidney, N. Shutty, and C. Jones, “Magic state cultivation: growing T states as cheap as CNOT gates,” *arXiv preprint arXiv:2409.17595*, 2024.
- [18] C. Gidney, M. Newman, P. Brooks, and C. Jones, “Yoked Surface Codes,” *Nature Communications*, vol. 16, no. 1, p. 4498, 2025, doi: [10.1038/s41467-025-59714-1](https://doi.org/10.1038/s41467-025-59714-1).
- [19] D. Litinski, “A Game of Surface Codes: Large-Scale Quantum Computing with Lattice Surgery,” *Quantum*, vol. 3, p. 128, 2019, doi: [10.22331/q-2019-03-05-128](https://doi.org/10.22331/q-2019-03-05-128).
- [20] M. Beverland, V. Kliuchnikov, and E. Schoute, “Surface Code Compilation via Edge-Disjoint Paths,” *PRX Quantum*, vol. 3, no. 2, p. 20342, 2022, doi: [10.1103/PRXQuantum.3.020342](https://doi.org/10.1103/PRXQuantum.3.020342).
- [21] J. van de Wetering, “ZX-calculus for the Working Quantum Computer Scientist,” no. arXiv:2012.13966. arXiv, 2020.

- [22] C. Gidney, “Stim: A Fast Stabilizer Circuit Simulator,” *Quantum*, vol. 5, p. 497, 2021, doi: [10.22331/q-2021-07-06-497](https://doi.org/10.22331/q-2021-07-06-497).
- [23] Z. Feng, Z. Liu, F. Lu, and N. Wang, “Quon Classical Simulation: Unifying Clifford, Matchgates and Entanglement,” *arXiv preprint arXiv:2505.07804*, 2025.

-  Link to the source code of this slides.
-  Link to TQEC Github repository.
-  Link to list of resources to learn more about this topic.